



**UNIVERSITÀ
DEGLI STUDI
DI BERGAMO**

Dipartimento di
Ingegneria Gestionale, dell'Informazione e della Produzione

Corso di Laurea in
Ingegneria Informatica

Classe L-8

Integrazione di eBPF in ambiente Android

Sfide e opportunità per l'ispezione di
sistema

Candidato:

Mattia Ferrari

Matricola n. 1083721

Relatore:

Prof. Matthew Rossi

ANNO ACCADEMICO

2024/2025

Sommario

Questa tesi affronta il problema dell'utilizzo di eBPF e bpfttrace per il monitoraggio di applicazioni Android a livello kernel. Android, pur basandosi sul kernel Linux, non rende disponibili nativamente gli strumenti di tracing tipici dei sistemi Linux, rendendo necessari adattamenti specifici per poterli utilizzare.

La soluzione proposta si basa sull'esecuzione di bpfttrace all'interno di una distribuzione Debian installata nell'emulatore Android Cuttlefish, in esecuzione su Ubuntu. Questo ambiente consente di osservare gli eventi del kernel Android tramite eBPF senza modificare il sistema operativo e senza richiedere hardware fisico.

Il sistema sviluppato comprende sette probe bpfttrace parallele per il monitoraggio di syscall, transazioni IPC Binder, ciclo di vita dei processi e traffico di rete, coordinate da un modulo di orchestrazione in Python e analizzate da un modulo post-sessione.

Le sperimentazioni su tre scenari, la navigazione web con Chrome, l'uso della galleria e della fotocamera, e l'attivazione della modalità aereo, mostrano che il monitoraggio parallelo delle probe permette di ricostruire il comportamento interno del sistema operativo in modo non ottenibile da una singola fonte di dati. I risultati evidenziano l'architettura di isolamento del renderer Chromium, la firma Binder specifica dell'accesso all'hardware fotografico e la sequenza di disattivazione e riconnessione dello stack WiFi.

Indice

1	Introduzione	1
1.1	Scenario applicativo	1
1.2	Criticità	2
1.3	Approccio proposto	3
2	Tecnologie di base	5
2.1	Linux	5
2.2	Android Cuttlefish	6
2.3	Tracing, eBPF e bpftrace	8
2.3.1	Tracing delle applicazioni	8
2.3.2	Tracing mediante eBPF	9
2.3.3	bpftrace	10
3	Implementazione	13
3.1	Architettura generale	13
3.1.1	Le probe bpftrace	14
3.1.2	Il modulo di orchestrazione	15
3.1.3	Il modulo di analisi	15
3.1.4	L'interfaccia utente	15
3.2	Considerazioni progettuali	15
3.3	Le probe	16
3.3.1	Ciclo di vita dei processi	16
3.3.2	Monitoraggio delle system call	18
3.3.3	Monitoraggio IPC Binder	19

3.3.4	Probe di rete	20
3.4	L'orchestratore	22
3.4.1	Gestione del processo bpftrace	22
3.4.2	Elaborazione degli eventi	22
3.4.3	Gestione della sessione e terminazione	24
3.5	Il launcher interattivo	24
3.5.1	Modalità operative	24
3.6	Il modulo di analisi	25
3.6.1	Analisi generale degli eventi	25
3.6.2	Analisi delle syscall e della latenza	26
3.6.3	Analisi del Binder IPC	26
3.6.4	Analisi di rete	26
3.6.5	Albero dei processi e resource map	27
3.6.6	Output	27
3.7	Il formato dei dati: JSON Lines	28
4	Test e Validazione	31
4.1	Attivazione della modalità aereo	31
4.1.1	Fase preliminare	32
4.1.2	Attivazione della modalità aereo	32
4.1.3	Disattivazione e riconnessione	33
4.1.4	Osservazioni sull'architettura osservata	33
4.2	Navigazione web con Chrome	34
4.2.1	Fase preliminare	34
4.2.2	Invio della query	36
4.2.3	Caricamento di google.com	36
4.2.4	Ricerca "google news"	37
4.2.5	Osservazioni sull'architettura osservata	37
4.3	Apertura della galleria e scatto fotografico	38
4.3.1	Apertura della galleria	38
4.3.2	Attivazione della fotocamera	40

4.3.3	Scatto della foto	40
4.3.4	Osservazioni sull'architettura osservata	41
5	Limiti e possibili miglioramenti	43
5.1	Limiti del sistema attuale	43
5.1.1	Dipendenza dall'ambiente emulato	43
5.1.2	Analisi esclusivamente post-sessione	44
5.1.3	Identificazione dei processi	44
5.2	Direzioni di sviluppo	44
5.2.1	Estensione a dispositivi reali	44
5.2.2	Analisi in tempo reale e sistema di allerta	45
5.2.3	Firme comportamentali e rilevamento di anomalie	45
5.2.4	Estensione delle probe	46
5.2.5	Interfaccia di visualizzazione	46
6	Conclusioni	47
	Bibliografia	49

Capitolo 1

Introduzione

1.1 Scenario applicativo

Negli ultimi anni i dispositivi mobili hanno assunto un ruolo centrale nell'ambito dell'informatica personale e professionale. Tra i sistemi operativi per dispositivi mobili, Android rappresenta una delle piattaforme più diffuse a livello mondiale [24] e viene utilizzato in una grande varietà di dispositivi, dagli smartphone ai sistemi embedded. La complessità crescente delle applicazioni e del sistema operativo rende sempre più importante disporre di strumenti in grado di analizzare e comprendere il comportamento delle applicazioni durante l'esecuzione.

Il monitoraggio delle attività delle applicazioni è fondamentale in diversi ambiti, tra cui il *debugging* del software, l'analisi delle prestazioni e la sicurezza informatica. In particolare, la possibilità di osservare le interazioni tra le applicazioni e il sistema operativo consente di individuare anomalie di funzionamento, colli di bottiglia prestazionali e potenziali comportamenti malevoli.

Tradizionalmente il *tracing* del sistema operativo viene realizzato mediante strumenti specifici che permettono di raccogliere informazioni sull'esecuzione dei processi e sulle attività del kernel. Tali strumenti possono tuttavia introdurre un significativo *overhead* computazionale oppure richiedere modifiche al sistema operativo, rendendone difficile l'utilizzo in ambienti reali. Inoltre, molti strumenti di analisi disponibili nei sistemi operativi tradizionali, e in particolare in ambiente Linux, non sono direttamente utilizzabili su Android senza adattamenti specifici.

Nel 2014 è stato introdotto l'extended Berkeley Packet Filter (eBPF) in Linux [15], una tecnologia che permette di eseguire piccoli programmi in modo sicuro ed efficiente all'interno del sistema operativo. Questo consente di osservare il comportamento delle applicazioni e del sistema senza la necessità di modifiche al codice sorgente o dello sviluppo di moduli dedicati. Per queste ragioni eBPF è diventato uno strumento sempre più utilizzato per attività di tracing e osservabilità nei sistemi Linux [13].

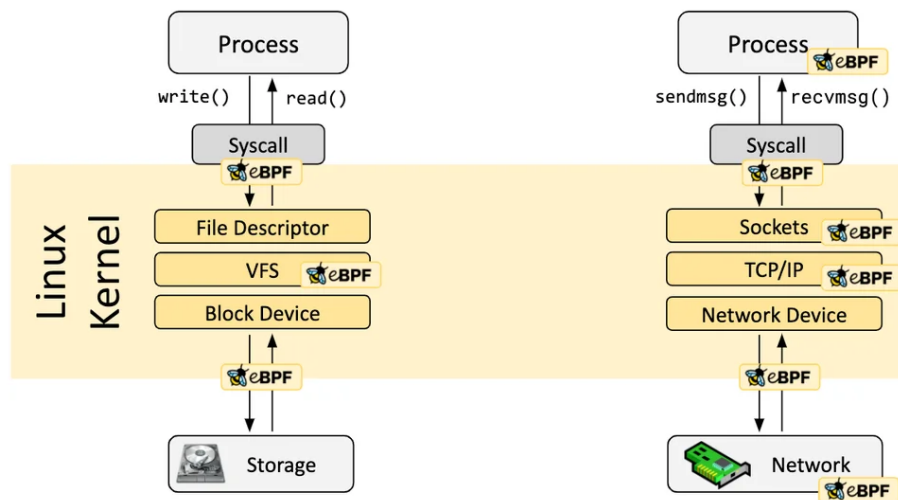


Figura 1.1: Architettura di eBPF nel kernel Linux [13]

1.2 Criticità

Tra gli strumenti che permettono di utilizzare eBPF in modo pratico, `bpftool` costituisce una soluzione particolarmente efficace per il tracing delle applicazioni e del sistema operativo. `bpftool` [9] mette a disposizione un linguaggio di *scripting* ad alto livello che consente di definire sonde e azioni associate agli eventi osservati, permettendo di raccogliere informazioni durante l'esecuzione senza sviluppare programmi eBPF completi. Questo approccio semplifica notevolmente la realizzazione di strumenti di analisi dinamica.

Tuttavia, l'utilizzo di `bpftool` in ambiente Android presenta diverse difficoltà pratiche. In molti dispositivi l'accesso alle funzionalità di basso livello del sistema è

limitato e spesso non sono disponibili gli strumenti e le librerie necessari per l'esecuzione di programmi basati su eBPF. Inoltre, l'installazione diretta di tali strumenti sul sistema Android può risultare complessa o non praticabile, soprattutto al di fuori di ambienti di sviluppo, quando non si ha accesso a certi privilegi che consentono di effettuare l'installazione e l'uso di tali strumenti. Ottenere tali permessi richiederebbe di effettuare il *rooting* del dispositivo o di sostituire la *release* Android installata, operazioni spesso non praticabili in contesti reali.

	Emulatore	Dispositivo di Produzione
Accesso root	Disponibile	Richiede rooting
Installazione strumenti	Possibile	Richiede rooting
Modifica della ROM	Non necessaria	Spesso necessaria
Utilizzo di bpftime	Fattibile	Non praticabile
Riproducibilità	Alta	Bassa

Tabella 1.1: Confronto tra ambiente di test e dispositivo di produzione Android.

Queste limitazioni rendono difficile l'impiego delle tecnologie basate su eBPF per l'analisi del comportamento delle applicazioni Android, nonostante il loro potenziale per il *monitoring* del sistema. Risulta quindi necessario definire un ambiente di lavoro che consenta l'utilizzo di bpftime in modo stabile e riproducibile, permettendo allo stesso tempo l'osservazione delle attività delle applicazioni durante l'esecuzione.

Il problema affrontato in questa tesi consiste quindi nel rendere possibile l'utilizzo di bpftime per il tracing delle applicazioni in ambiente Android, individuando una configurazione che consenta di superare le limitazioni tipiche del sistema e di ottenere dati significativi sull'esecuzione delle applicazioni.

1.3 Approccio proposto

Per superare le difficoltà legate all'utilizzo degli strumenti basati su eBPF in ambiente Android, in questo lavoro è stata sviluppata una soluzione che permette di eseguire lo strumento bpftime in un ambiente controllato e riproducibile, mantenendo

do allo stesso tempo la possibilità di osservare il comportamento delle applicazioni Android.

La soluzione proposta si basa sulla creazione di un ambiente di sviluppo virtualizzato in cui sia possibile eseguire Android e utilizzare strumenti tipicamente disponibili nei sistemi Linux tradizionali. A questo scopo è stato utilizzato l'emulatore Android Cuttlefish [4], eseguito su sistema operativo Ubuntu, che consente di disporre di un ambiente Android completo e configurabile per attività di sviluppo e sperimentazione.

All'interno dell'ambiente Android è stata installata una distribuzione Linux minimale basata su Debian [27]. Questa distribuzione è stata utilizzata come ambiente di lavoro per l'esecuzione di bpftrace e degli strumenti necessari al tracing. L'utilizzo di una distribuzione Linux separata ha permesso di evitare la compilazione nativa di bpftrace su Android e di semplificare l'installazione delle dipendenze necessarie.

In questo modo è stato possibile realizzare un ambiente che combina la flessibilità degli strumenti Linux con la possibilità di osservare il comportamento di applicazioni eseguite su Android. Su questa base è stato sviluppato un insieme di script e strumenti software che permettono di eseguire sessioni di tracing e di raccogliere automaticamente i dati prodotti da bpftrace.

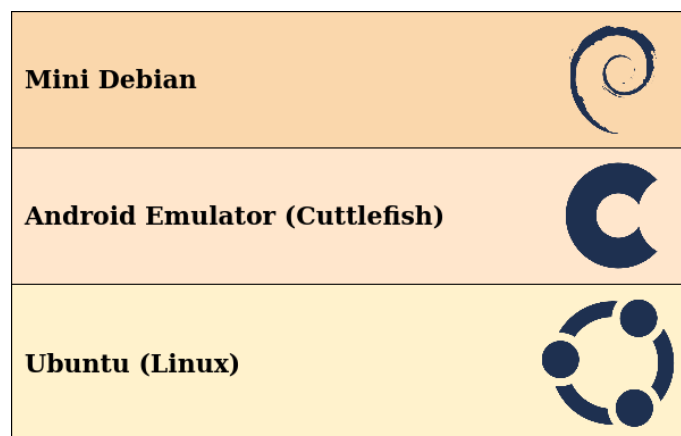


Figura 1.2: Architettura della soluzione proposta: Ubuntu, Cuttlefish e ambiente Debian.

Capitolo 2

Tecnologie di base

2.1 Linux

Linux è una famiglia di sistemi operativi *open-source Unix-like*, basati sull'omonimo kernel, ampiamente utilizzato sia in ambito accademico sia industriale [29] grazie alla sua stabilità, flessibilità e natura open source. Il kernel gestisce le risorse hardware e fornisce ai programmi un'interfaccia standardizzata attraverso le system call, mentre l'insieme dei programmi in spazio utente costituisce l'ambiente operativo vero e proprio. Ciò che rende Linux particolarmente adatto allo sviluppo software e alle attività di sperimentazione è la sua natura open source e free: il codice sorgente è liberamente accessibile, modificabile e distribuibile, il che consente un controllo dettagliato del sistema e facilita l'accesso a un ecosistema ampio di strumenti di analisi e debugging.

In questo lavoro è stato utilizzato Ubuntu 22.04 LTS, una distribuzione Linux ampiamente diffusa. Ubuntu è stato scelto per la disponibilità di pacchetti software aggiornati e per la semplicità di configurazione dell'ambiente di sviluppo. Il sistema Ubuntu ha costituito la piattaforma principale su cui sono stati installati gli strumenti necessari allo svolgimento della tesi, tra cui l'ambiente di virtualizzazione Android Cuttlefish e i componenti utilizzati per il tracing basato su eBPF. L'utilizzo di una distribuzione Linux standard ha permesso di disporre di un ambiente stabile e riproducibile, facilitando l'installazione delle dipendenze e la gestione degli strumenti software impiegati nelle sperimentazioni.

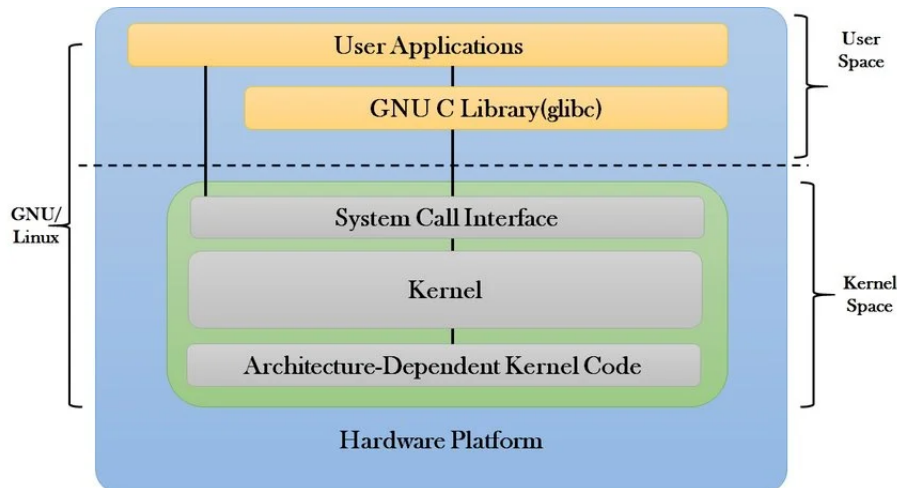


Figura 2.1: Architettura del sistema Linux: kernel space, user space e system call [21].

2.2 Android Cuttlefish

Android [16] è un sistema operativo per dispositivi mobili basato sul kernel Linux e progettato per supportare l'esecuzione di applicazioni in un ambiente isolato e controllato. Il sistema è organizzato in una struttura a livelli che comprende il kernel Linux, le librerie di sistema, il *runtime* Android, il *framework* delle applicazioni e il livello delle applicazioni utente. Il kernel Linux costituisce il livello più basso dell'architettura e si occupa della gestione delle risorse hardware, della memoria, dei processi, dei dispositivi di input/output e delle comunicazioni di rete. I livelli superiori forniscono invece i servizi necessari all'esecuzione delle applicazioni e definiscono l'ambiente software tipico della piattaforma Android.

Ogni applicazione Android viene eseguita in uno spazio isolato rispetto alle altre applicazioni, generalmente associato a un utente distinto. Questo meccanismo di isolamento contribuisce alla sicurezza del sistema ma limita l'accesso diretto alle risorse e alle funzionalità di basso livello. Inoltre, molte componenti tipiche delle distribuzioni Linux tradizionali non sono presenti oppure risultano fortemente ridotte, rendendo più complesso l'utilizzo di strumenti di analisi progettati per ambienti Linux standard.

Nonostante Android utilizzi il kernel Linux, l'ambiente software differisce in modo significativo da quello di una distribuzione Linux tradizionale. In particolare,

l'organizzazione del file system, la disponibilità delle librerie e la gestione dei permessi rendono spesso necessario un adattamento degli strumenti sviluppati per sistemi Linux convenzionali. Questo aspetto è particolarmente rilevante nel caso degli strumenti di tracing, che richiedono generalmente l'accesso a funzionalità di basso livello del sistema operativo.

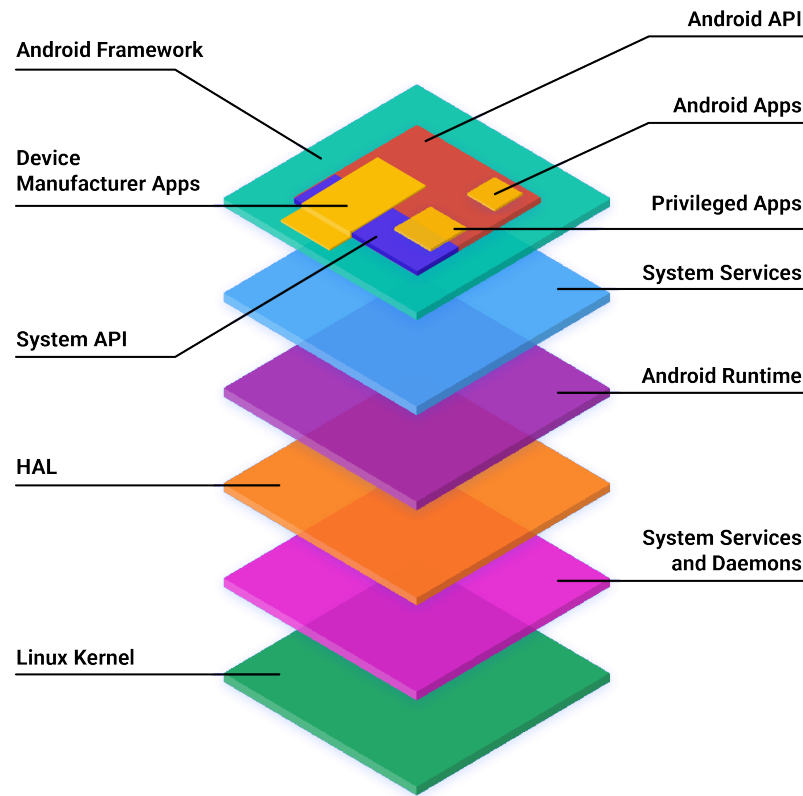


Figura 2.2: Architettura a livelli del sistema operativo Android [1].

Per lo sviluppo e la sperimentazione è stato utilizzato Android Cuttlefish [4], un emulatore ufficiale della piattaforma Android progettato per l'esecuzione su sistemi Linux. Cuttlefish esegue il sistema Android all'interno di una macchina virtuale che riproduce i principali componenti di un dispositivo reale, inclusi kernel, sistema operativo e servizi applicativi, senza la necessità di hardware fisico.

In questo progetto l'emulatore è stato eseguito su Ubuntu, che ha costituito la piattaforma principale per l'installazione e la configurazione degli strumenti di tracing. L'ambiente virtualizzato ha permesso di controllare la configurazione del sistema e di ripetere le sperimentazioni in modo sistematico.

2.3 Tracing, eBPF e bpftrace

2.3.1 Tracing delle applicazioni

Il tracing delle applicazioni è una tecnica di analisi che consiste nella raccolta sistematica di informazioni durante l'esecuzione di un programma o di un sistema operativo, con l'obiettivo di osservare il comportamento dinamico del software e le sue interazioni con il sistema. A differenza dell'analisi statica, che si basa sull'esame del codice sorgente o dei file eseguibili, il tracing consente di studiare ciò che avviene realmente durante l'esecuzione, permettendo di ottenere informazioni temporali sugli eventi generati dal sistema.

Nel contesto dei sistemi operativi moderni, il tracing può essere effettuato a diversi livelli di osservazione. A livello applicativo è possibile monitorare le operazioni effettuate da un processo, come la creazione di nuovi processi, l'accesso ai file o le comunicazioni di rete. A livello di sistema operativo è invece possibile osservare eventi gestiti dal kernel, come le chiamate di sistema, la schedulazione dei processi o le operazioni di input/output. Questa possibilità di osservare eventi a diversi livelli rende il tracing uno strumento fondamentale per comprendere il funzionamento complessivo di un sistema software.

Le tecniche di tracing vengono utilizzate principalmente per tre scopi. Il primo è il debugging del software, dove l'osservazione dettagliata delle operazioni eseguite da un programma permette di individuare errori o comportamenti inattesi. Il secondo è l'analisi delle prestazioni, in cui il tracing consente di identificare colli di bottiglia e di comprendere come le risorse del sistema vengono utilizzate dalle applicazioni. Il terzo ambito è quello della sicurezza informatica, dove l'osservazione delle attività dei processi permette di individuare comportamenti anomali o potenzialmente malevoli.

Tradizionalmente il tracing in ambiente Linux è stato realizzato tramite strumenti basati su meccanismi come `ptrace` [20], utilizzati ad esempio da programmi come `strace` [25], oppure tramite infrastrutture di tracing integrate nel kernel come `ftrace` [23] e `perf` [19]. Questi strumenti permettono di ottenere informazioni dettagliate sul comportamento del sistema, ma possono risultare complessi da utilizzare oppure introdurre un overhead significativo durante l'esecuzione.

2.3.2 Tracing mediante eBPF

Una delle tecnologie più recenti per il tracing nei sistemi Linux è rappresentata dall'extended Berkeley Packet Filter (eBPF) [15]. eBPF permette di eseguire piccoli programmi all'interno del sistema operativo in modo controllato e sicuro, consentendo di intercettare eventi durante l'esecuzione senza modificare il codice del kernel o delle applicazioni.

Il funzionamento di eBPF si basa sull'inserimento dinamico di programmi che vengono eseguiti quando si verificano determinati eventi, come l'ingresso in una funzione del kernel o l'attivazione di un *tracepoint*. Prima di essere caricati, i programmi eBPF vengono verificati da un componente del sistema operativo chiamato *verifier*, che ne controlla la sicurezza e garantisce che non possano compromettere la stabilità del sistema. Questo meccanismo permette di eseguire codice in modo sicuro anche all'interno di parti critiche del sistema operativo.

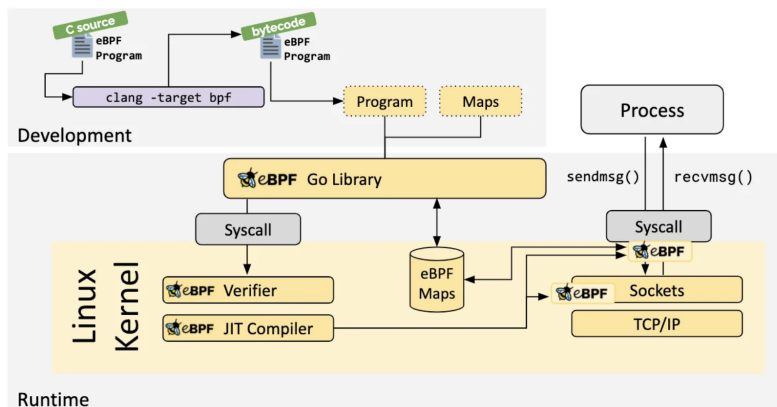


Figura 2.3: Flusso di caricamento ed esecuzione di un programma eBPF nel kernel Linux [13].

Rispetto alle tecniche di tracing tradizionali, eBPF presenta alcune caratteristiche distintive. In primo luogo consente di raccogliere informazioni con un overhead generalmente ridotto, poiché i programmi vengono eseguiti direttamente nel contesto dell'evento osservato. Inoltre, le sonde possono essere attivate, disattivate e ridefinite dinamicamente senza riavviare o modificare il sistema operativo, il che permette di adattare l'osservazione alle proprie esigenze senza alcun impatto sul sistema monitorato.

Grazie a queste caratteristiche eBPF rappresenta oggi una tecnologia particolarmente adatta per il tracing dinamico dei sistemi Linux e costituisce la base di numerosi strumenti moderni di osservabilità.

2.3.3 bpftrace

Per utilizzare eBPF in modo pratico sono stati sviluppati diversi strumenti software. Tra questi, **bpftrace** [9] rappresenta uno degli strumenti più diffusi per la realizzazione di script di tracing basati su eBPF.

bpftrace è un linguaggio di scripting e un interprete che permette di definire programmi di tracing in forma compatta e leggibile. Gli script vengono tradotti automaticamente in programmi eBPF ed eseguiti dal sistema operativo, semplificando notevolmente lo sviluppo rispetto alla scrittura diretta di codice eBPF.

Un programma **bpftrace** è costituito da una o più *probe*, cioè punti di osservazione associati a eventi specifici. Ogni probe è composta da due parti principali:

- la definizione dell'evento da osservare
- le azioni da eseguire quando l'evento si verifica

La struttura generale di una probe è illustrata in Figura 2.4, che evidenzia la separazione tra la definizione dell'evento da osservare e il blocco delle azioni da eseguire.

```
1 tracepoint:sched:sched_process_exec
2 {
3     printf("{\"ts_ns\": \"%llu\", \"ts\": \"%s\", \"type\": \"process\",
4           \"event\": \"exec\", \"pid\": %d, \"tid\": %d, \"comm\": \"%s\"}\n\",
5           nsecs, strftime(\"%H:%M:%S\", nsecs), pid, tid, comm);
6 }
```

Definizione dell'evento

Blocco azioni

Figura 2.4: Struttura di una probe bpftrace: definizione dell'evento e blocco delle azioni.

bpftrace supporta diversi tipi di probe, che permettono di osservare eventi a diversi livelli del sistema. Nel presente lavoro sono stati utilizzati esclusivamente i tracepoint, in quanto rappresentano la soluzione più stabile e compatibile tra diverse versioni del kernel.

I tracepoint sono punti di osservazione statici definiti all'interno del sistema operativo. A differenza di altre tipologie di probe, la loro interfaccia è stabile tra diverse versioni del kernel, il che li rende particolarmente adatti a un ambiente come Android, dove la versione del kernel può variare. Ogni tracepoint espone un insieme strutturato di argomenti che descrivono l'evento osservato, consentendo di accedere direttamente a informazioni quali il processo coinvolto, i parametri della chiamata e il risultato dell'operazione. I tracepoint disponibili nel sistema utilizzato durante le sperimentazioni sono illustrati in Figura 2.5.

```
tracepoint:binder:binder_transaction
tracepoint:binder:binder_transaction_alloc_buf
tracepoint:binder:binder_transaction_received
tracepoint:raw_syscalls:sys_enter
tracepoint:raw_syscalls:sys_exit
tracepoint:sched:sched_process_exec
tracepoint:sched:sched_process_exit
tracepoint:sched:sched_process_fork
tracepoint:sched:sched_switch
tracepoint:sched:sched_wakeup
tracepoint:sched:sched_wakeup_new
```

Figura 2.5: Lista dei tracepoint presenti nell'ambiente Android Cuttlefish e utilizzati durante le sperimentazioni.

bpftrace supporta anche altri due tipi di probe. Le **kprobe** consentono di inserire sonde dinamiche all'ingresso o all'uscita di funzioni del kernel, offrendo grande flessibilità anche in punti privi di tracepoint predefiniti; la loro stabilità è tuttavia inferiore, poiché dipendono dai nomi interni delle funzioni del kernel che possono variare tra versioni diverse. Le **uprobe** operano invece in spazio utente e permettono di tracciare funzioni di librerie o applicazioni specifiche senza modificarne il codice, risultando utili quando l'obiettivo è osservare il comportamento interno di un singolo processo piuttosto che eventi di sistema.

Capitolo 3

Implementazione

Il codice sorgente del progetto è disponibile pubblicamente su GitHub [14].

3.1 Architettura generale

Il sistema è composto da quattro componenti principali che collaborano secondo un'architettura a *pipeline*: il livello di raccolta basato su probe **bpfttrace** raccoglie eventi direttamente dal kernel, il modulo di orchestrazione li normalizza e persiste, il modulo di analisi li elabora in fase post-sessione, e l'interfaccia utente orchestra l'intera interazione. L'architettura complessiva è illustrata in Figura 3.1.

Il flusso di dati segue un percorso ben definito: le probe producono eventi strutturati in formato JSON su stdout, il modulo di orchestrazione li consuma riga per riga e li persiste in un file `.jsonl`, e il modulo di analisi legge questo file al termine della sessione per produrre *report* e statistiche aggregate. L'interfaccia utente funge da punto di ingresso unico che coordina l'avvio delle probe, la durata della sessione e la generazione del report finale.

Questa separazione delle responsabilità risponde a un principio architetturale preciso: ciascun componente è progettato per fare una sola cosa e farlo bene, rimanendo testabile e utilizzabile in modo indipendente. Le interfacce di comunicazione tra i componenti sono deliberatamente semplici, ovvero stdout/stdin per il flusso di eventi e file JSON per la configurazione e la persistenza, in modo da minimizzare le dipendenze e facilitare la manutenzione.

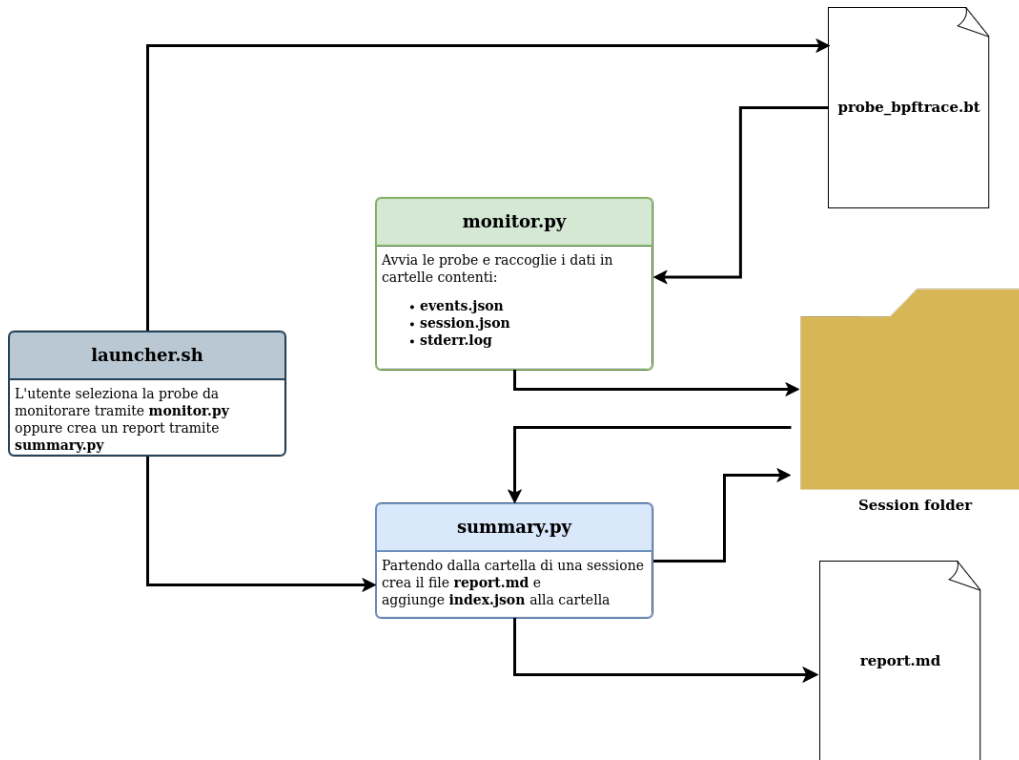


Figura 3.1: Architettura del sistema: flusso di dati tra il livello di raccolta (probe bpfttrace), il modulo di orchestrazione, il modulo di analisi il tutto all'interno dell'interfaccia utente

3.1.1 Le probe bpfttrace

Le probe operano interamente in spazio kernel, con accesso diretto alle strutture dati del kernel quali `task_struct` [28], e producono output in formato JSON su stdout. La scelta di JSON come formato di output permette a bpfttrace di comportarsi come un produttore di eventi strutturati che il livello superiore può consumare senza logica di parsing ad hoc. Ogni evento emesso segue uno schema comune con i campi: `ts_ns` (timestamp in nanosecondi), `ts` (orario leggibile), `type` (categoria dell'evento), `event` (nome specifico dell'evento), `pid`, `ppid`, `tid`, `uid`, `comm` (nome del processo, massimo 15 caratteri imposto dal kernel) e un oggetto `data` contenente i campi specifici della probe.

3.1.2 Il modulo di orchestrazione

Il modulo di orchestrazione è il cuore del sistema. Si occupa di avviare bpfftrace come sottoprocesso, gestire il ciclo di vita della sessione di raccolta dati, normalizzare e validare gli eventi ricevuti, arricchirli dove necessario tramite il filesystem `/proc` e salvarli su disco in formato JSON Lines (`.jsonl`). Opera come *consumer* dello *stream* stdout di bpfftrace, consumando gli eventi riga per riga in modo sincrono, mentre un thread dedicato drena contemporaneamente stderr per separare gli errori diagnostici.

3.1.3 Il modulo di analisi

Il modulo di analisi è responsabile dell'elaborazione post-sessione. Legge il file degli eventi prodotto dal modulo di orchestrazione, calcola statistiche aggregate su syscall, processi, Binder IPC e attività di rete, e genera un report testuale. Il modulo produce inoltre un file `index.json` all'interno della directory di sessione, contenente tutti i dati analitici in forma strutturata.

3.1.4 L'interfaccia utente

L'interfaccia utente è implementata come script bash e costituisce il punto di ingresso per l'utente. Offre un menù interattivo a colori che permette di scegliere quale probe eseguire, per quanto tempo, e se generare automaticamente un report al termine della sessione. Supporta anche una modalità non interattiva tramite flag da riga di comando (`-p` per specificare la probe, `-t` per il timer).

3.2 Considerazioni progettuali

Alcune scelte progettuali meritano una riflessione esplicita. La separazione tra il livello di raccolta basato su probe bpfftrace (che produce dati grezzi) e il modulo di orchestrazione (che normalizza, arricchisce e persiste) risponde a un principio di separazione delle responsabilità: bpfftrace è ottimizzato per la raccolta ad alta frequenza di eventi in spazio kernel, mentre Python è più adatto per la logica ap-

plicativa complessa come la lettura di `/proc` o la gestione delle sessioni. Un'unica componente che facesse entrambe le cose sarebbe più fragile e difficile da mantenere.

La scelta di correlare `sys_enter` e `sys_exit` nelle probe di rete (invece di usare solo `sys_enter`) aggiunge la dimensione temporale all'osservazione: non solo si sa che una syscall è stata invocata, ma anche quanto ha impiegato e se ha avuto successo. Questa informazione è preziosa per l'analisi delle prestazioni e per il rilevamento di anomalie basato su latenza.

La suddivisione dell'attività di rete in quattro probe separate (invece di una sola probe che monitora tutte le syscall di rete) è una scelta di modularità che permette all'utente di attivare solo il sottoinsieme di informazioni di cui ha bisogno, riducendo il volume di dati prodotti e l'overhead sul sistema monitorato.

Infine, la presenza di un'interfaccia utente dedicata come *wrapper* bash (invece di integrare la gestione del timer direttamente nel modulo di orchestrazione) riflette la filosofia Unix di strumenti semplici e componibili [22]: il modulo di orchestrazione si occupa solo di monitorare, l'interfaccia utente si occupa dell'orchestrazione e dell'esperienza utente, e il modulo di analisi si occupa solo dell'elaborazione post-sessione. Questa separazione rende ciascun componente testabile e utilizzabile indipendentemente.

3.3 Le probe

Il progetto include sette probe bpftrace, ognuna dedicata a osservare una specifica classe di eventi di sistema. Ogni probe è implementata come script bpftrace indipendente; la loro configurazione (metadati, codice identificativo, tipo di evento atteso) è centralizzata in un registro condiviso consultato sia dal modulo di orchestrazione che dall'interfaccia utente.

3.3.1 Ciclo di vita dei processi

Questa probe monitora le tre fasi fondamentali del ciclo di vita di un processo: la creazione (*fork*), l'esecuzione di un nuovo programma (*exec*) e la terminazione (*exit*).

Per farlo, aggancia tre tracepoint dello scheduler del kernel Linux: `sched:sched_process_fork`, `sched:sched_process_exec` e `sched:sched_process_exit`.

La scelta di questi tracepoint è motivata dalla loro stabilità e dalla ricchezza di informazioni che espongono. I tracepoint `sched_process_*` sono presenti e stabili in tutte le versioni del kernel Linux utilizzate da Android, e forniscono direttamente i metadati del processo coinvolto senza necessità di accedere a strutture kernel aggiuntive. Rispetto all'alternativa di intercettare le syscall `fork` o `execve` tramite `raw_syscalls`, questi tracepoint vengono attivati in un momento del ciclo di vita del processo in cui le strutture dati del kernel sono già aggiornate e consistenti, riducendo il rischio di osservare stati intermedi.

Un aspetto rilevante è la raccolta del `ppid` (Process ID del processo padre), ottenuto tramite un cast esplicito alla struttura kernel `task_struct`: il campo `real_parent->tgid` fornisce l'identificativo del gruppo di thread del processo genitore, informazione fondamentale per ricostruire l'albero genealogico dei processi. Il campo `comm`, recuperato direttamente dal kernel, è soggetto al limite di 15 caratteri imposto dal kernel Linux; per questa ragione, il modulo di orchestrazione arricchisce gli eventi di tipo `exec` leggendo il file `/proc/<pid>/cmdline` prima che il processo termini, ottenendo così la riga di comando completa.

Gli eventi generati da questa probe sono utilizzati dal modulo di analisi per costruire l'albero dei processi e correlare l'attività dei diversi componenti del sistema Android.

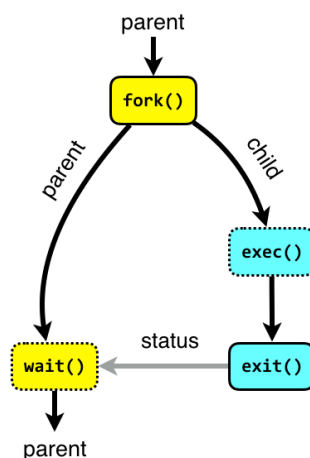


Figura 3.2: Diagramma del ciclo di vita di un processo Linux [8]

3.3.2 Monitoraggio delle system call

La probe di monitoraggio delle syscall utilizza il tracepoint generico `raw_syscalls:sys_enter`, che viene attivato all'ingresso di *qualsiasi* system call invocata nel sistema. Per limitare l'osservazione alle sole syscall di interesse, la probe applica un filtro inline sul campo `args->id`, che contiene il numero identificativo della syscall, ad esempio:

- `execve` (id 59): esecuzione di un nuovo programma;
- `openat` (id 257): apertura di un file;
- `connect` (id 42): tentativo di connessione di rete.

Questo approccio, ovvero intercettare tutto e filtrare, è preferibile rispetto all'uso di tracepoint specifici per syscall (come `tracepoint:syscalls:sys_enter_execve`) perché consente di gestire tutte le syscall di interesse con un unico *handler*, riducendo la duplicazione del codice e semplificando la gestione delle mappe temporanee condivise.

La probe implementa una decodifica semantica degli argomenti per ciascuna syscall. Per `execve` viene letto il percorso del binario da eseguire; per `openat` il percorso del file da aprire; per `connect` viene ispezionata la struttura `sockaddr` per determinare la famiglia di indirizzi (`AF_UNIX`, `AF_INET` o `AF_INET6`) e decodificare l'indirizzo di destinazione.

Un aspetto critico da considerare nella lettura di questi parametri è il problema del *time-of-check to time-of-use* (TOCTOU) [7]. I percorsi dei file letti da `execve` e `openat`, così come gli indirizzi letti dalla struttura `sockaddr` in `connect`, risiedono nella memoria del processo utente. Nel momento in cui la probe li legge tramite `uptr()`, il processo potrebbe in teoria modificarli prima che il kernel li utilizzi effettivamente. Questo significa che il valore osservato dalla probe potrebbe non corrispondere al valore effettivamente usato dal kernel per eseguire l'operazione. Per scopi di tracing comportamentale e analisi forense questo margine di incertezza è generalmente accettabile, ma è importante tenerne conto quando si interpretano i dati

raccolti, in particolare in contesti di sicurezza dove un attaccante potrebbe sfruttare questa finestra temporale.

Per evitare memory leak nelle mappe bpfttrace utilizzate come variabili temporanee indicizzate per `tid`, la probe include un handler del tracepoint `sched:sched_process_exit` che cancella i valori residui, e un handler `END` che pulisce le mappe al termine.

3.3.3 Monitoraggio IPC Binder

Binder [2] è il meccanismo di Inter-Process Communication (IPC) fondamentale di Android. Ogni chiamata a un servizio di sistema, ogni interazione tra applicazioni e framework Android, ogni chiamata a un Content Provider passa attraverso Binder. Monitorare Binder significa quindi avere visibilità sul tessuto connettivo dell'intero sistema operativo Android.

La scelta di usare i tracepoint del driver Binder è obbligata: non esistono syscall dirette che esponano le transazioni Binder in modo strutturato, poiché tutta la comunicazione avviene tramite il driver `/dev/binder` attraverso `ioctl`. I tracepoint `binder:binder_transaction_*` sono l'unico punto di osservazione che fornisce informazioni strutturate sulle transazioni a livello kernel, senza necessità di intercettare e decodificare le `ioctl` manualmente.

La probe aggancia tre tracepoint esposti dal driver kernel del Binder:

- `binder:binder_transaction`: generato all'avvio di una transazione, cattura mittente, destinatario e metadati della chiamata;
- `binder:binder_transaction_received`: generato quando il destinatario riceve la transazione, permette di correlare invio e ricezione tramite il campo `debug_id`;
- `binder:binder_transaction_alloc_buf`: generato quando viene allocato il buffer dati, fornisce la dimensione del payload trasferito.

Il tracepoint `binder_transaction` espone tra i suoi argomenti un campo `flags` che viene decodificato per determinare se la transazione è one-way (asincrona, bit

0x01 dei flag impostato) o sincrona. Il `ppid` è recuperato tramite cast a `task_struct` come nelle altre probe.

La correlazione tra i tre eventi tramite `debug_id` è fondamentale: solo incrociando `binder_transaction` (che ha i dati del mittente), `binder_transaction_alloc_buf` (che ha la dimensione del payload) e `binder_transaction_received` (che ha i dati del destinatario) è possibile ricostruire un'immagine completa di ogni transazione IPC. Questa correlazione viene eseguita dal modulo di analisi nella fase post-sessione.

```
{
  "ts_ns": "3345836913100", "ts": "10:06:56", "type": "ipc", "event": "binder_transaction_received",
  "pid": 615, "tid": 1218, "uid": 1000, "comm": "binder:615_5", "data": {"debug_id": 285240},
  "schema_version": 1, "probe_code": "P00011"}

{"ts_ns": "3345853536799", "ts": "10:06:57", "type": "ipc", "event": "binder_transaction", "pid":
525, "ppid": 1, "tid": 700, "uid": 1000, "comm": "_thread", "data": {"debug_id": 285241,
"target_node": 999, "to_pid": 615, "to_tid": 0, "reply": 0, "oneway": 1, "code": 4, "flags": 17},
"schema_version": 1, "probe_code": "P00011"}

{"ts_ns": "3345853557843", "ts": "10:06:57", "type": "ipc", "event":
"binder_transaction_alloc_buf", "pid": 525, "tid": 700, "uid": 1000, "comm": "_thread", "data":
{"debug_id": 285241, "data_size": 144, "offsets_size": 0, "extra_buffers_size": 0},
"schema_version": 1, "probe_code": "P00011"}

{"ts_ns": "3345853574933", "ts": "10:06:57", "type": "ipc", "event": "binder_transaction_received",
"pid": 615, "tid": 1218, "uid": 1000, "comm": "binder:615_5", "data": {"debug_id": 285241},
"schema_version": 1, "probe_code": "P00011"}

{"ts_ns": "3345870220738", "ts": "10:06:57", "type": "ipc", "event": "binder_transaction", "pid":
525, "ppid": 1, "tid": 700, "uid": 1000, "comm": "_thread", "data": {"debug_id": 285242,
"target_node": 999, "to_pid": 615, "to_tid": 0, "reply": 0, "oneway": 1, "code": 4, "flags": 17},
"schema_version": 1, "probe_code": "P00011"}
```

Figura 3.3: Schema della correlazione tra i tre tracepoint Binder tramite il campo `debug_id`.

3.3.4 Probe di rete

L'attività di rete è monitorata tramite quattro probe specializzate, ciascuna dedicata a una syscall diversa. La scelta di osservare le syscall di rete tramite il tracepoint generico `raw_syscalls:sys_enter/sys_exit`, filtrato per numero di syscall, invece di tracepoint specifici è motivata dalla necessità di correlare ingresso e uscita della stessa syscall per calcolare la latenza e leggere il valore di ritorno. Questa tecnica di correlazione richiede di salvare il timestamp e il contesto all'ingresso in una mappa indicizzata per `tid`, e di recuperarli all'uscita. La suddivisione in probe separate è

inoltre una scelta progettuale che offre flessibilità: è possibile attivare solo il monitoraggio delle connessioni in uscita, o solo il volume di dati inviati, o solo i servizi in ascolto, senza attivare l'intero set.

Anche per le probe di rete vale la considerazione TOCTOU già discussa per la probe di monitoraggio delle syscall: gli indirizzi letti dalla struttura `sockaddr` in memoria utente potrebbero in teoria essere modificati dal processo tra il momento della lettura da parte della probe e il momento in cui il kernel utilizza effettivamente il valore.

Probe di invio dati Traccia la syscall `sendto` (id 44). Utilizza la tecnica di correlazione `sys_enter/sys_exit`: all'ingresso della syscall vengono salvate in mappe `bpftrace` indicizzate per `tid` le informazioni di contesto (timestamp, `ppid`, file descriptor, dimensione richiesta); all'uscita viene calcolata la latenza come differenza di timestamp in nanosecondi e convertita in microsecondi. Il valore di ritorno `args->ret` corrisponde al numero di byte effettivamente inviati, che può differire dalla dimensione richiesta in caso di invio parziale.

Probe di ricezione dati Funziona in modo simmetrico rispetto alla probe di invio dati ma aggancia `recvfrom` (id 45). Registra la dimensione del buffer allocato per la ricezione e il numero di byte effettivamente ricevuti, oltre alla latenza della syscall.

Probe di associazione socket Traccia `bind` (id 49), la syscall con cui un processo richiede al kernel di associare un socket a un indirizzo locale, tipicamente per mettersi in ascolto su una porta. La probe decodifica la struttura `sockaddr` all'ingresso della syscall, distinguendo `AF_INET` (indirizzi IPv4, con decodifica di porta e indirizzo tramite `ntop()`) e `AF_UNIX` (socket locali, con lettura del percorso). Solo i `bind` andati a buon fine (valore di ritorno uguale a 0) vengono evidenziati nell'analisi come porte effettivamente aperte.

Probe di accettazione connessioni Traccia `accept` (id 43) e `accept4` (id 288), le syscall con cui un processo accetta una connessione in ingresso. Il peer address vie-

ne letto dalla struttura `sockaddr` popolata dal kernel all'uscita della syscall, quando il descrittore della nuova connessione è già disponibile. In fase di analisi, i processi con $\text{UID} \geq 10000$ (UID delle applicazioni Android) che invocano `accept` vengono segnalati come anomali, poiché su Android i processi applicativi normalmente non agiscono da server.

Una caratteristica comune a tutte e quattro le probe di rete è la pulizia delle mappe temporanee nel blocco `END`, necessaria per evitare che le strutture dati in memoria nel kernel rimangano allocate al termine del programma `bpfttrace`.

3.4 L'orchestratore

Il modulo di orchestrazione è il componente che coordina l'intero ciclo di vita di una sessione di monitoraggio. All'avvio legge il registro delle probe, verifica che la probe richiesta sia presente, crea una directory di sessione sotto `sessions/` con nome composto dal codice della probe e dal timestamp corrente (ad esempio `syscalls_2025-01-15_10-30-00`), e salva immediatamente un file `session.json` con i metadati della sessione.

3.4.1 Gestione del processo `bpfttrace`

`bpfttrace` viene avviato come sottoprocesso tramite `subprocess.Popen` con `stdout` e `stderr` come pipe separate, in modalità line-buffered. Questa separazione è essenziale: `stdout` trasporta esclusivamente gli eventi JSON emessi dalle probe tramite `printf`, mentre `stderr` contiene i messaggi diagnostici di `bpfttrace` (errori di compilazione, avvisi sui tracepoint, messaggi di avvio). Un thread separato legge `stderr` e lo scrive su un file `stderr.log` all'interno della directory di sessione, garantendo che il thread principale non si blocchi mai in attesa di dati su `stderr`.

3.4.2 Elaborazione degli eventi

Il thread principale itera riga per riga sullo `stdout` di `bpfttrace`. Per ogni riga non vuota tenta il parsing JSON; le righe che non sono JSON valido vengono considerate

output diagnostico e reindirizzate su `stderr.log`. Gli eventi JSON validi vengono processati attraverso una pipeline di tre fasi:

Normalizzazione Garantisce che ogni evento rispetti lo schema minimo atteso. Se il campo `schema_version` manca, viene aggiunto con il valore definito nella configurazione della probe. Se i campi `type` o `event` mancano, vengono aggiunti come fallback dalla configurazione. Se il campo `data` non è presente o non è un oggetto, viene inizializzato come dizionario vuoto. Viene inoltre aggiunto il campo `probe_code` per facilitare la correlazione fuori dalla cartella di sessione.

Arricchimento Attivo solo per gli eventi di tipo `exec`. Legge `/proc/<pid>/cmdline` per ottenere la riga di comando completa del processo appena eseguito (i valori sono separati da caratteri null, che vengono sostituiti con spazi). Legge inoltre il link simbolico `/proc/<pid>/exe` per ottenere il percorso assoluto dell'eseguibile. Questo arricchimento compensa il limite di 15 caratteri del campo `comm` nel kernel e aggiunge informazioni non accessibili direttamente da `bpfttrace` in un tracepoint.

Validazione Confronta i campi `type` ed `event` dell'evento ricevuto con i valori attesi dalla configurazione della probe. Le discrepanze generano avvisi su `stderr.log` ma non bloccano la scrittura dell'evento, evitando perdita di dati a fronte di probe multi-evento (indicate con `event: "*" nella configurazione`).

Ogni evento normalizzato, arricchito e validato viene serializzato nuovamente in JSON e scritto su `events.jsonl` con flush immediato, garantendo che i dati vengano persistiti su disco in tempo reale anche in caso di interruzione improvvisa del processo.



Figura 3.4: Pipeline di elaborazione degli eventi nel modulo di orchestrazione

3.4.3 Gestione della sessione e terminazione

Alla ricezione del segnale di interruzione (Ctrl-C, gestito tramite un blocco `try/except` su `KeyboardInterrupt`), il modulo di orchestrazione esegue la pulizia ordinata: invia `SIGTERM` al processo `bpfttrace` e, se questo non termina entro un breve intervallo, forza la terminazione tramite `SIGKILL`, aggiorna il file `session.json` con il timestamp di fine sessione e stampa un riepilogo a terminale. La directory di sessione finale contiene quindi: `events.jsonl` (gli eventi raccolti), `stderr.log` (output diagnostico di `bpfttrace`) e `session.json` (metadati con orari di inizio e fine).

3.5 Il launcher interattivo

L'interfaccia utente costituisce il punto di ingresso principale per l'utente ed è implementata come script `bash`. All'avvio esegue una serie di controlli preliminari verificando la presenza di `python3`, `bpfttrace`, `jq` e `timeout` nel `PATH`, e la disponibilità del modulo di orchestrazione e del registro delle probe. In caso di errore viene mostrato un messaggio diagnostico chiaro e lo script termina con codice di uscita 1.

La lista delle probe disponibili viene costruita dinamicamente leggendo le chiavi di `probes_map.json` tramite `jq`, e filtrata tenendo solo le probe il cui file `.bt` esiste effettivamente su disco. Questa lettura dinamica garantisce che il menù sia sempre sincronizzato con lo stato reale del filesystem, senza necessità di aggiornamenti manuali.

3.5.1 Modalità operative

Lo script supporta tre modalità operative accessibili dal menù principale:

Avvio probe Permette di selezionare una probe (con selezione numerica) e un timer in secondi. La probe viene eseguita e, tramite il comando `timeout`, invia `SIGTERM` al modulo di orchestrazione allo scadere del tempo. Durante l'esecuzione viene mostrata una barra di avanzamento ASCII animata che aggiorna stato e percentuale ogni secondo.

Generazione report Mostra una lista delle sessioni disponibili in `sessions/`, con il numero di eventi raccolti e il timestamp di avvio (letto da `session.json`). L'utente sceglie la sessione e lo script invoca il modulo di analisi per generare il report.

Full monitoring Combina avvio probe e generazione report in sequenza automatica. Dopo la conclusione di ogni probe, il launcher localizza la directory di sessione appena creata estraendo il percorso dall'output del modulo di orchestrazione tramite `grep`, con un fallback che cerca la directory più recente in `sessions/`, e invoca immediatamente il modulo di analisi su di essa.

La variabile d'ambiente `PYTHONUNBUFFERED=1` viene impostata prima di avviare il modulo di orchestrazione per forzare lo svuotamento immediato del buffer stdout di Python, evitando che gli output vengano persi quando `timeout` termina il processo prima che i buffer vengano svuotati.

3.6 Il modulo di analisi

Il modulo di analisi è il componente responsabile dell'elaborazione post-sessione. Accetta come argomento il percorso di una directory di sessione, legge i file degli eventi e dei metadati di sessione, calcola un insieme di metriche aggregate e produce sia un report testuale che un file `index.json` con tutti i dati analitici in forma strutturata.

3.6.1 Analisi generale degli eventi

La prima fase di analisi calcola statistiche generali: il numero totale di eventi, la loro distribuzione per tipo (`syscall`, `process`, `ipc`, `network`) e per nome specifico (`execve`, `openat`, `fork`, `binder_transaction`, ecc.), e i processi più attivi per volume di eventi generati. Viene inoltre calcolato il tasso di eventi al secondo, sia tramite i metadati di sessione (`started_at` e `stopped_at` da `session.json`) sia tramite il campo `ts_ns` degli eventi stessi, che permette di identificare il picco di attività al secondo e la finestra temporale più intensa.

3.6.2 Analisi delle syscall e della latenza

Per gli eventi di tipo `syscall` vengono calcolati conteggi per nome, numero di errori (rilevati tramite valore di ritorno negativo) e tasso di errore percentuale. Se gli eventi includono il campo `lat_us` (latenza in microsecondi, presente nelle probe che correlano `sys_enter` e `sys_exit`), vengono calcolati i percentili p50, p95 e p99 sia globalmente che per nome di syscall e per processo. I 20 eventi più lenti vengono elencati separatamente, e viene prodotta una distribuzione dei codici errno più frequenti, sia globale che per singola syscall.

3.6.3 Analisi del Binder IPC

L'analisi Binder richiede la correlazione di tre tipi di eventi distinti. Il modulo di analisi costruisce due indici in memoria: uno indicizzato per `debug_id` sugli eventi `binder_transaction_alloc_buf` (per recuperare la dimensione del payload) e uno sugli eventi `binder_transaction_received` (per recuperare il `comm` del processo destinatario). Per ogni evento `binder_transaction` non di tipo `reply`, viene cercata la corrispondenza nei due indici, costruendo così una visione completa della transazione con mittente, destinatario, dimensione dati e tipo (one-way vs sincrono).

Dal grafo delle transazioni viene costruito un IPC graph che mappa gli archi di comunicazione tra processi, con il numero di chiamate e i byte trasferiti per ogni coppia mittente-destinatario. Questo grafo è il punto di partenza per analisi comportamentali: pattern insoliti di comunicazione, processi che si scambiano volumi anomali di dati, o servizi di sistema contattati da applicazioni non attese.

3.6.4 Analisi di rete

L'analisi di rete aggrega i dati delle quattro probe in un quadro coerente. Vengono costruiti: il port landscape (elenco delle porte effettivamente aperte, filtrato sui bind con `ret == 0`), la lista dei processi che hanno accettato connessioni in ingresso, il volume di dati inviati e ricevuti per processo, e una timeline al secondo dei byte trasmessi. La timeline viene utilizzata per identificare il picco di invio e il picco di ricezione. I processi con `UID ≥ 10000` che agiscono da server tramite `accept`

```
IPC communication graph (top 10 edges):
_thread → binder:615_3: 2882 calls, 415008 bytes
_thread → binder:615_1: 2411 calls, 347184 bytes
_thread → binder:615_5: 2063 calls, 297072 bytes
surfaceflinger → binder:525_2: 1678 calls, 735576 bytes
_thread → binder:615_2: 1119 calls, 161136 bytes
NetworkService → binder:8406_5: 1031 calls, 347560 bytes
m.webview_shell → binder:8406_5: 1027 calls, 282332 bytes
m.webview_shell → binder:615_3: 758 calls, 86644 bytes
Compositor → binder:8373_8: 749 calls, 564672 bytes
NetworkService → binder:8406_3: 743 calls, 259460 bytes
```

Figura 3.5: Esempio di output del report Binder IPC con grafo delle transazioni tra processi.

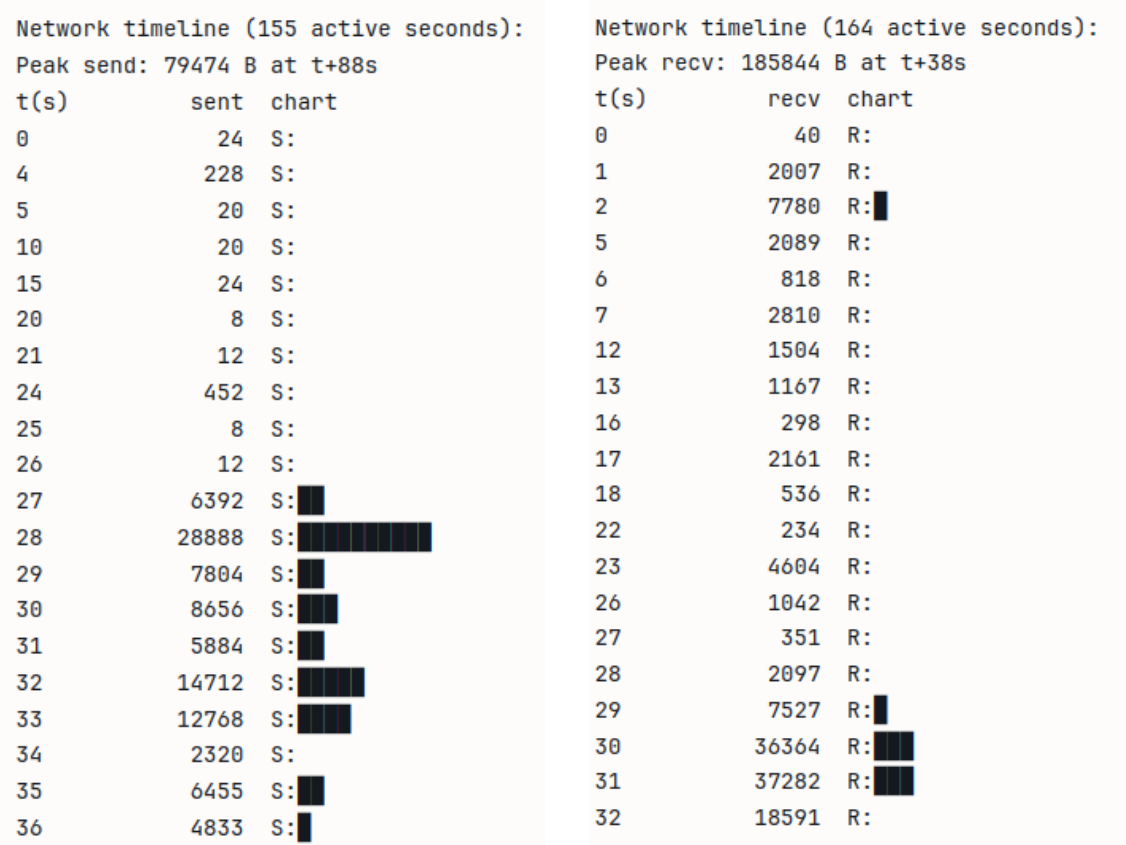
vengono segnalati separatamente come potenzialmente anomali, poiché su Android le applicazioni normalmente non aprono porte in ascolto.

3.6.5 Albero dei processi e resource map

A partire dai campi `pid` e `ppid` presenti in tutti gli eventi, il modulo di analisi ricostruisce l'albero genealogico dei processi osservati durante la sessione. La struttura ad albero viene serializzata sia in forma gerarchica (per la visualizzazione) che in forma flat (per la ricerca rapida). La resource map aggrega invece, per ogni processo, l'insieme dei file aperti (da eventi `openat`), degli indirizzi di rete contattati (da eventi `connect`) e dei binari eseguiti (da eventi `execve`), fornendo una sorta di fingerprint comportamentale per ciascun processo osservato.

3.6.6 Output

Il modulo di analisi produce tre output distinti. Il file `index.json` nella directory di sessione contiene tutti i dati analitici in forma strutturata e machine-readable, pensato per elaborazioni successive o visualizzazioni esterne. Il report testuale, stampato su stdout e salvato in `reports/summaries/` con il nome della sessione come identificativo, presenta le stesse informazioni in forma leggibile, con sezioni separate per ogni categoria di analisi e una barra ASCII per la timeline di rete.



(a) Timeline del traffico in uscita

(b) Timeline del traffico in entrata

Figura 3.6: Esempio di output del report di analisi di rete

3.7 Il formato dei dati: JSON Lines

Tutti gli eventi prodotti dal sistema vengono persistiti [17] in formato JSON Lines [18] (`.jsonl`), dove ogni riga è un oggetto JSON indipendente e completo. Questo formato è stato scelto per diverse ragioni pratiche: supporta il parsing incrementale riga per riga senza necessità di caricare l'intero file in memoria, è compatibile con strumenti di analisi standard come `jq` [11], `pandas` [30] e `Apache Spark` [6], e permette lo streaming in tempo reale poiché ogni evento è autonomo e non dipende dal contesto degli eventi precedenti.

Lo schema comune a tutti gli eventi include i campi di intestazione `ts_ns`, `ts`, `type`, `event`, `pid`, `ppid`, `tid`, `uid`, `comm`, e un oggetto `data` con i campi specifici della probe. Il campo `ts_ns`, espresso in nanosecondi di uptime del kernel, garantisce una risoluzione temporale sufficiente per la correlazione di eventi rapidi come le

```

Process tree:
init (pid 1)
├── ueventd (pid 102)
├── servicemanager (pid 221)
├── netd (pid 513)
│   ├── iptables-restore (pid 518)
│   └── ip6tables-restore (pid 519)
├── main (pid 516)
│   ├── CpuMonitorService (pid 776)
│   ├── NetworkMonitor/ (pid 1046)
│   ├── webview_zygote (pid 1123)
│   │   ├── VideoFrameCompo (pid 8406)
│   │   ├── Le.GoogleSearch (pid 1650)
│   │   ├── trigger_perfett (pid 8333)
│   │   ├── ThreadPoolForeg (pid 8373)
│   │   └── trigger_perfett (pid 8727)
│   ├── adbd (pid 629)
│   │   ├── cut (pid 8000)
│   │   └── ... (33 cut processes)
│   ├── traced (pid 651)
│   ├── bpftrace (pid 5923)
│   ├── android.hardware (pid 7974)
│   └── ... (40+ android.hardware / flags_health_ch pairs)
├── proot (pid 5305)
│   ├── bash (pid 5306)
│   │   ├── bash (pid 8022)
│   │   │   ├── jq (pid 8046)
│   │   │   │   ├── ... (20+ jq processes)
│   │   │   └── timeout (pid 8127)
│   │   │       ├── python3 (pid 8131)
│   │   │       │   ├── bpftrace (pid 8134)
│   │   │       │   └── sh (pid 8140)
│   │   └── bash (pid 8129)
│   │       ├── sleep (pid 8130)
│   │       └── ... (120+ sleep processes)
│   └── python3 (pid 9484)
├── proot (pid 5320..5376) [5 istanze]
├── bpftrace (pid 7829, 7863, 7910, 7935)
│   └── sh (pid 7950, 7952)
└── sleep (pid 7947..9521) [~500 istanze orfane]

```

(a) Albero dei processi

```

Resource map (files / IPs / binaries per process):
.quicksearchbox:
[files] /proc/self/stat
[files] /sys/devices/system/cpu/possible
ActivityManager:
[files] /sys/fs/cgroup/uid_99000
[files] /sys/fs/cgroup/uid_99000/.
[files] /sys/fs/cgroup/uid_99000/cgroup.controllers
[files] /sys/fs/cgroup/uid_99000/cgroup.events
[files] /sys/fs/cgroup/uid_99000/cgroup.freeze
[files] ... (30 total)
AsyncTask #1:
[files] /sys/devices/system/cpu/possible
AsyncTask #2:
[files] /sys/devices/system/cpu/possible
AudioOut_D:
[files] /proc/605/task/735/sched
AudioThread:
[files] /dev/ashmem4a0eb604-6525-49b9-a0ae-7f0748f6a4f5
CachedAppOptimi:
[files] /proc/1159/status
[files] /proc/1228/status
[files] /proc/1290/status
[files] /proc/1488/status
[files] /proc/1556/status
[files] ... (41 total)
ChildProcessMai:
[files] /dev/__properties__/u:object_r:timezone_prop:s0
[files] /proc/meminfo
[files] /proc/self/stat
[files] /proc/self/status
[files] /product/app/webview/webview.apk
Chrome_ChildIOT:
[files] /dev/ashmem4a0eb604-6525-49b9-a0ae-7f0748f6a4f5
Chrome_IOTThread:
[files] /dev/ashmem4a0eb604-6525-49b9-a0ae-7f0748f6a4f5

```

(b) Resource map

Figura 3.7: Esempio di output del report

transazioni Binder. Il campo **ts**, invece, fornisce un orario leggibile in formato HH:MM:SS, utile per la visualizzazione nei report.

Capitolo 4

Test e Validazione

In questo capitolo vengono presentati i risultati delle sessioni di monitoraggio effettuate sull'emulatore Android Cuttlefish. Per ciascuno scenario, le sette probe del sistema sono state eseguite in parallelo: la probe delle syscall per il monitoraggio di `openat`, `connect` ed `execve`; la probe del Binder per le transazioni IPC; la probe del ciclo di vita dei processi per la nascita e terminazione; la probe di associazione socket e la probe di accettazione connessioni per il monitoraggio dei socket in ascolto e delle connessioni in ingresso; la probe di invio dati e la probe di ricezione dati per il volume di dati inviati e ricevuti.

Gli scenari analizzati sono tre: l'attivazione della modalità aereo, una sessione di navigazione web con Chrome, e l'apertura dell'applicazione galleria con accesso alla fotocamera.

4.1 Attivazione della modalità aereo

L'applicazione monitorata è `com.android.settings` (uid 1000), il pannello di configurazione di sistema di Android. Lo scenario consiste nell'attivare e poi disattivare la modalità aereo tramite il menu delle impostazioni rapide. Il riferimento temporale è l'avvio della probe del Binder, con le altre sei probe avviate contestualmente. I due eventi chiave sono l'attivazione della modalità aereo e la sua disattivazione.

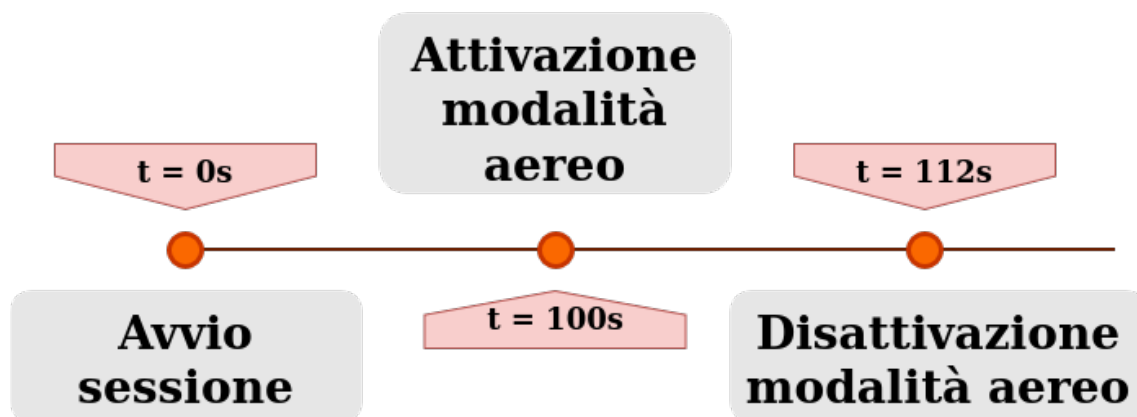


Figura 4.1: Sequenza delle azioni nella sessione di attivazione e disattivazione della modalità aereo.

4.1.1 Fase preliminare

Nei 100 secondi precedenti all'attivazione, il sistema opera nella schermata home con il WiFi attivo. La probe del Binder registra un'attività di fondo dominata dal canale chiamato da `surfaceflinger`, con 3.330 chiamate per circa 1.4 MB: è il compositor grafico [5] che aggiorna continuamente la schermata.

4.1.2 Attivazione della modalità aereo

All'attivazione della modalità aereo si produce un effetto immediato e netto sullo stack di rete. La probe di associazione socket non registra alcun evento tra durante l'intera finestra che copre il periodo in modalità aereo e la successiva riattivazione non genera alcun `bind` da processi di rete. È la conferma diretta che lo stack WiFi è stato completamente fermato.

Il percorso del comando attraverso il sistema è ricostruibile dalle probe. L'utente interagisce con `com.android.settings`, che invia via Binder [2] il *broadcast* di sistema `ACTION_AIRPLANE_MODE_CHANGED` al `ConnectivityService` all'interno del *system server*. È quest'ultimo, non l'applicazione impostazioni, a coordinare la sequenza di disattivazione: WiFi e Bluetooth vengono fermati attraverso chiamate ai rispettivi HAL via Binder. Il processo `ndroid.settings` accumula 4.169 transazioni Binder nell'intera sessione, ma nessuna di esse produce direttamente traffico di rete: tutto passa attraverso il *system server*.

4.1.3 Disattivazione e riconnessione

Alla disattivazione della modalità aereo viene avviata una sequenza di riconnessione osservabile passo per passo attraverso le probe. Il primo segnale misurabile arriva con un po' di ritardo rispetto al toggle: vengono registrate le prime operazioni `bind` dopo la pausa. Successivamente `IpClient.wlan0` apre sei nuovi socket per avviare la negoziazione DHCP, e `netd` crea un thread per la risoluzione dei nomi. Il ritardo riflette il tempo necessario al *firmware* dell'interfaccia WiFi virtuale di Cuttlefish per completare la riassociazione alla rete prima che lo stack di rete superiore possa riattivarsi.

La probe di ricezione dati mostra che `IpClient.wlan0` riceve complessivamente circa 229 KB nell'intera sessione: è il traffico DHCP [12] corrispondente all'acquisizione dell'indirizzo IP dopo la riconnessione.

Il picco di Binder dell'intera sessione (4.099 eventi/s) si registra due secondi prima del primo bind di rete. Esso è prodotto dai *broadcast* di `ConnectivityService` che notificano tutti i processi del sistema del cambiamento di stato della rete, prima ancora che il collegamento fisico sia ristabilito. Anche questa sequenza, il Binder che esplode prima che i socket di rete si riaprano, è osservabile solo grazie al monitoraggio parallelo delle sette probe.

4.1.4 Osservazioni sull'architettura osservata

Lo scenario della modalità aereo si distingue rispetto ai prossimi due per la natura dell'evento monitorato: non si tratta dell'avvio di un'applicazione ma di un cambiamento di stato del sistema che coinvolge l'infrastruttura di rete.

Il segnale più caratteristico è la finestra di silenzio nelle probe di rete. La probe di associazione socket non registra eventi, e la probe di accettazione connessioni mostra un gap analogo. Questa assenza prolungata di attività, in un sistema che normalmente produce `bind` e `accept` continuamente, è l'indicatore diretto della disattivazione dello stack WiFi.

Il traffico inviato via socket (2.8 MB totali) è dominato non dal WiFi ma dalle comunicazioni interne dell'hardware audio e grafico (`android.hardware`: 1.8 MB,

app SurfaceControl: 709 KB). Il contributo diretto di `wificond` è di soli 4.9 KB, segno che il controllo della radio WiFi avviene quasi interamente via Binder e non via socket.

Categoria	Volume totale
Transazioni Binder	35.058
Traffico IPC	11.3 MB
Dati inviati via socket	2.8 MB
Dati ricevuti via socket	4.8 MB
Bind durante modalità aereo	0

Tabella 4.1: Riepilogo del traffico complessivo della sessione modalità aereo: il silenzio nelle probe di rete durante la finestra $t \approx 92s-147s$ è il segnale diretto della disattivazione dello stack WiFi.

4.2 Navigazione web con Chrome

L'applicazione monitorata è `org.chromium.webview_shell` (uid 10064), un'applicazione di test che espone direttamente il motore di rendering WebView di Chromium. Lo scenario consiste nella navigazione web a partire dalla schermata home dell'emulatore: viene prima effettuata una ricerca tramite il widget di ricerca presente nella home, poi si naviga sul sito `google.com` e si esegue una ricerca di notizie. Le probe vengono avviate prima di qualsiasi interazione con il browser, per poter documentare il comportamento di base del sistema e distinguerlo dall'attività generata dall'utente.

La sessione si articola in quattro fasi, identificate in base alle azioni effettuate:

4.2.1 Fase preliminare

Nei primi 36 secondi l'emulatore si trova nella schermata home e non viene effettuata alcuna interazione. Questa fase serve a documentare il comportamento di base del sistema: Android non è silenzioso nemmeno in *idle*, e senza questa *baseline*

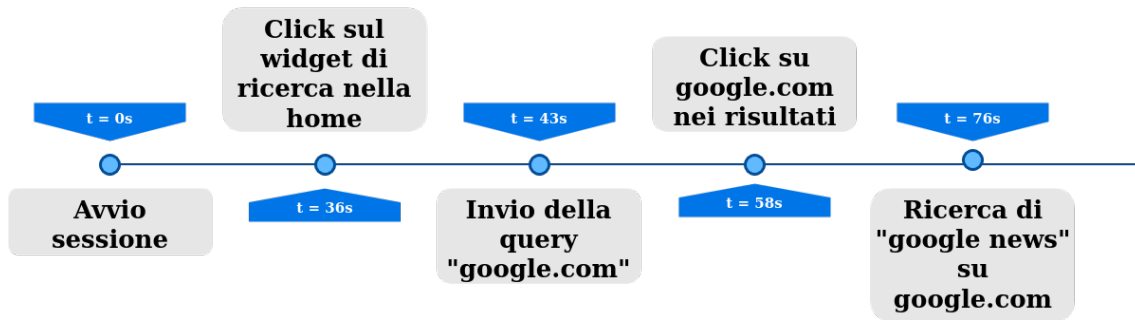


Figura 4.2: Sequenza delle azioni nella sessione di navigazione web.

sarebbe impossibile distinguere il traffico di fondo dall'attività riconducibile alle azioni utente.

La probe di ricezione dati mostra traffico già dai primi secondi, generato da `logcat` e da altri servizi di sistema che comunicano su socket locali in modo continuativo. Più interessante è un picco di invio che compare intorno a $t \approx 27s$, con il sistema che trasmette prima 6.4 KB poi 28.9 KB in due secondi consecutivi. Il traffico è dominato da `InputDispatcher` (18.4 KB a $t \approx 28s$) e dal processo `app` (`SurfaceControl`), con un contributo minore di `android.ui`; non ha nulla a che fare con il browser, che non è ancora avviato. Si tratta di comunicazioni interne al sottosistema grafico e di input, un esempio di quanto il sistema operativo produca traffico di rete per ragioni completamente indipendenti dall'utente.

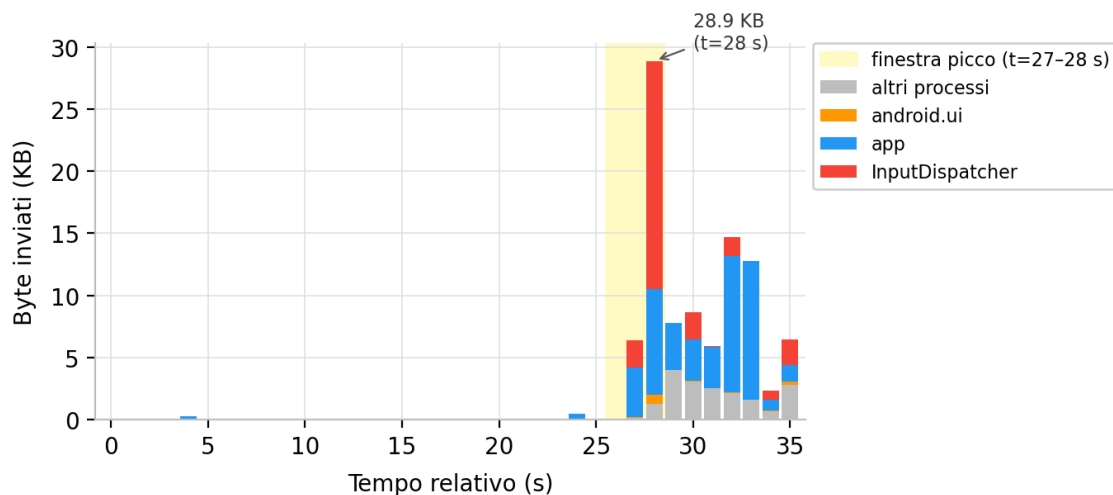


Figura 4.3: Picco di invio nella fase preliminare, generato dal compositor grafico prima di qualsiasi interazione utente.

4.2.2 Invio della query

Il click sul widget di ricerca non produce eventi netti nelle probe di rete. L'attività misurabile parte qualche secondo dopo, quando la probe delle syscall intercetta una `connect` da parte dell'applicazione Google Search che gestisce il widget nella schermata home. Contestualmente, la probe di accettazione connessioni mostra che `netd` crea un thread che nasce per gestire la sua richiesta DNS. È quindi l'app di ricerca Google, e non il browser, ad avviare la risoluzione del nome mentre l'utente digita la query.

La prima attività di rete del processo Chrome compare circa nove secondi dopo il click, quando le probe di invio e ricezione dati registrano i primi eventi dal thread `RenderThread` (PID 8373). I thread `m.webview_shell` e `NetworkService` compaiono nei secondi successivi. Il fatto che thread con nomi diversi condividano lo stesso PID è un effetto del limite di 15 caratteri del campo `comm` nel kernel Linux [28], che riporta il nome del singolo thread piuttosto che quello del processo.

Successivamente, la probe del Binder registra i primi eventi associati a `CrRenderer_Main` (PID 8406), il renderer di Chromium. La comunicazione tra `NetworkService` e il renderer avviene via Binder, non via socket: nell'arco dell'intera sessione vengono registrate 1031 transazioni tra i due per un totale di circa 339 KB. Ogni risposta HTTP ricevuta dalla rete viene consegnata al renderer attraverso questo canale IPC prima di essere visualizzata.

4.2.3 Caricamento di google.com

Il click su `google.com` produce in un secondo un *burst* di otto operazioni `bind` da parte di `NetworkService`, che indicano l'apertura di nuovi socket per il caricamento parallelo delle risorse della pagina, coerente con il *multiplexing* HTTP/2 [31] utilizzato da Chrome.

Il picco di connessioni in ingresso arriva con un ritardo di circa dodici secondi rispetto al click. Il ritardo è compatibile con il tempo di caricamento del contenuto dinamico della homepage di Google, che include numerose richieste secondarie per immagini, script e dati personalizzati.

4.2.4 Ricerca “google news”

La ricerca di “google news” produce la risposta più netta dell’intera sessione. La probe delle syscall registra il picco assoluto di 608 eventi al secondo, contro una media di circa 187 per l’intera sessione.

Il picco di invio dati segue con un ritardo di circa dodici secondi: la probe di invio dati registra 79.4 KB inviati in un secondo, il valore massimo dell’intera sessione. Il ritardo riflette il tempo che il browser impiega a ricevere i risultati, elaborarli tramite il renderer e inviare le richieste secondarie per i contenuti collegati.

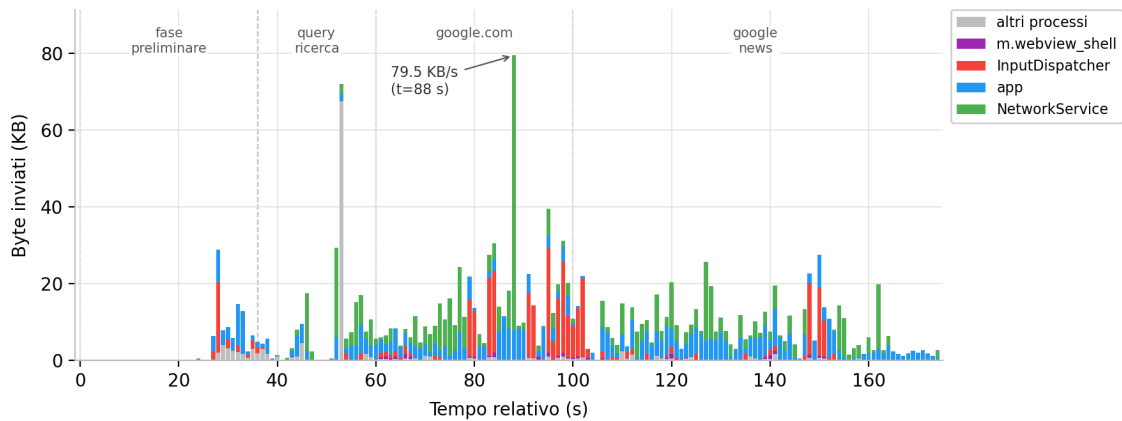


Figura 4.4: Picco di 79.4 KB inviati in un secondo, valore massimo dell’intera sessione.

4.2.5 Osservazioni sull’architettura osservata

Il monitoraggio parallelo delle sette probe ha permesso di osservare una catena di eventi che nessuna singola probe avrebbe potuto documentare per intero.

Il renderer `CrRendererMain` (PID 8406) non ha accesso diretto alla rete. Tutta la comunicazione tra il codice di rendering e lo stack di rete passa attraverso Binder: la catena osservata è `m.webview_shell` → Binder → `NetworkService` → socket. Questo isolamento è una scelta progettuale di Chromium [26] per separare il processo di rendering, potenzialmente esposto a contenuti non fidati, dal processo che gestisce le connessioni di rete.

Complessivamente, la sessione ha prodotto 42 343 transazioni Binder per un totale di circa 17 MB trasferiti via IPC, a fronte di 1.55 MB inviati e 2.37 MB ricevuti via socket. Il volume di traffico IPC interno supera quindi di circa cinque volte il traffico di rete esterno: anche una semplice sessione di navigazione web genera internamente un volume di comunicazione inter-processo di gran lunga superiore a quello scambiato con la rete.

Categoria	Volume totale
Transazioni Binder	42.343
Traffico IPC	17 MB
Dati inviati via socket	1.55 MB
Dati ricevuti via socket	2.37 MB

Tabella 4.2: Riepilogo del traffico complessivo della sessione Chrome: 42 343 transazioni Binder e confronto tra volume IPC e traffico di rete.

4.3 Apertura della galleria e scatto fotografico

L'applicazione monitorata è l'applicazione predefinita per la gestione delle immagini su Android (uid 10066). Lo scenario combina due applicazioni distinte: nella prima parte l'utente naviga la galleria fotografica, nella seconda passa alla fotocamera e scatta un'immagine. Il riferimento temporale è l'avvio della probe del Binder, con le altre sei probe avviate contestualmente nei secondi precedenti. Gli eventi si distribuiscono in tre momenti precisi: apertura della galleria, attivazione della fotocamera e scatto della foto.

4.3.1 Apertura della galleria

All'apertura dell'applicazione galleria, la probe del ciclo di vita dei processi registra la creazione del processo `droid.gallery3d` (PID 14805) come figlio del *system server* di Android. L'associazione all'uid 10066 è confermata dai percorsi dei file

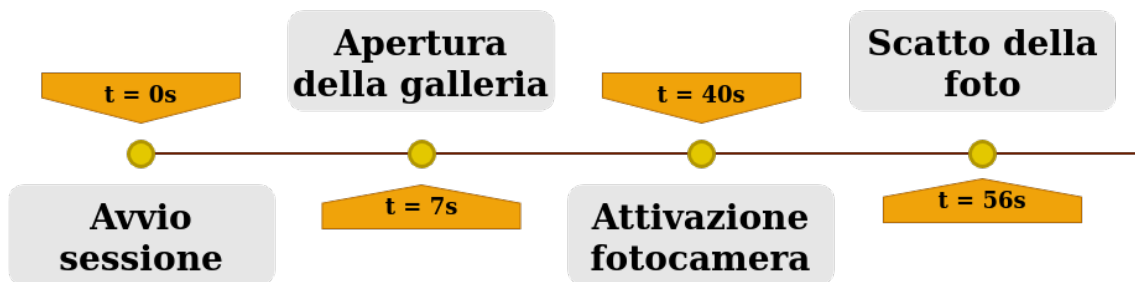


Figura 4.5: Sequenza delle azioni nella sessione di apertura galleria e fotocamera.

cgroup che il processo `ActivityManager` monitora per la gestione della memoria dell'applicazione.

La probe delle syscall rileva le prime operazioni di accesso allo storage: un thread dell'applicazione apre la radice dello spazio di archiviazione condiviso. Questo accesso corrisponde alla scansione iniziale del catalogo di immagini che la galleria esegue all'avvio per costruire la lista dei contenuti disponibili.

Sul fronte del Binder, l'avvio della galleria produce un'attività crescente che culmina nel picco assoluto della sessione: la probe registra 4 723 transazioni in un singolo secondo, circa cinque volte la media della sessione (941 eventi/s). Questo picco corrisponde al momento in cui la galleria carica le anteprime delle immagini, operazione che richiede un'intensa sequenza di comunicazioni con `SurfaceFlinger` [5] per il rendering di ciascun thumbnail.

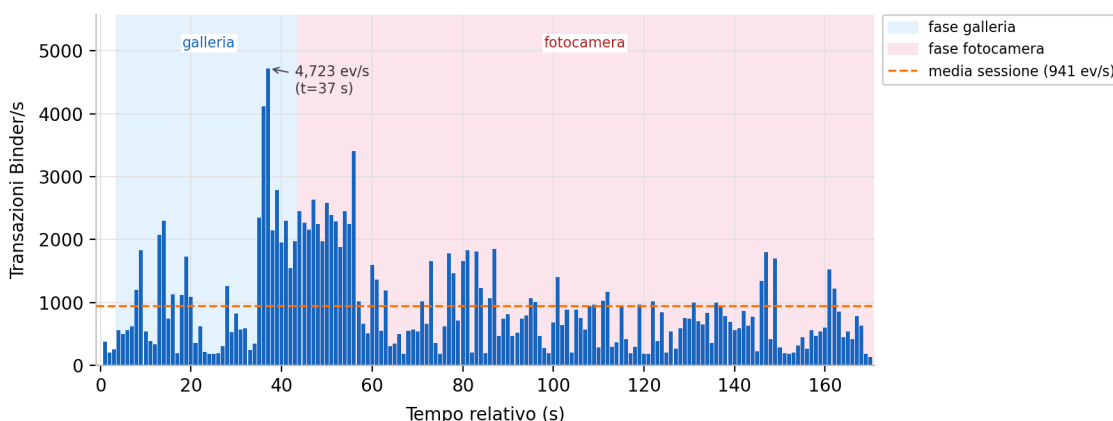


Figura 4.6: Picco di 4 723 transazioni Binder, durante il caricamento delle anteprime da parte della galleria.

4.3.2 Attivazione della fotocamera

Il passaggio alla fotocamera produce il cambiamento più caratteristico dell'intera sessione nel profilo delle comunicazioni Binder. Il processo `android.camera2` (PID 1913) diventa attivo e due thread legati all'hardware della fotocamera emergono tra i processi più trafficati: `Cam0_ResultDisp` con 6 285 eventi e `C3Dev-0-ReqQueue` con 4 280 eventi, il secondo e il quarto posto nell'intera sessione.

Il grafo IPC mostra che `Cam0_ResultDisp` comunica principalmente con il canale che corrisponde alla pipeline di elaborazione dei frame della fotocamera: il thread che riceve i risultati dal sensore li trasmette al `Camera Hardware Abstraction Layer` [3] attraverso Binder, che li consegna al processo di visualizzazione per l'anteprima in tempo reale. L'attivazione di questo canale è assente in tutti gli altri scenari analizzati e rappresenta quindi un segnale affidabile dell'accesso all'hardware della fotocamera.

In parallelo, `SurfaceFlinger` mantiene il canale più trafficato dell'intera sessione: 4 481 chiamate verso il `binder` per un totale di circa 2.1 MB. Questo traffico riflette la composizione continua dei frame dell'anteprima, che richiede una frequenza di comunicazione elevata tra il compositor grafico e il driver del display.

4.3.3 Scatto della foto

Lo scatto della fotografia lascia una traccia diretta nella resource map prodotta dalla probe delle syscall. Un thread del processo `droid.gallery3d` apre il file `IMG_20260304_113342.jpg`: si tratta dell'immagine appena acquisita, letta dall'applicazione per aggiornare il catalogo con il nuovo scatto. Lo stesso thread accede anche a `/share_history.xml`, il file che l'applicazione aggiorna sistematicamente a ogni modifica del catalogo.

La probe di ricezione dati registra circa 215 KB ricevuti dal processo `droid.gallery3d` su socket locali. Il ritardo di circa tredici secondi rispetto allo scatto è compatibile con il tempo necessario al sistema di gestione dei media di Android per processare il nuovo file, generare l'anteprima e notificare l'applicazione dell'avvenuto aggiornamento del catalogo.

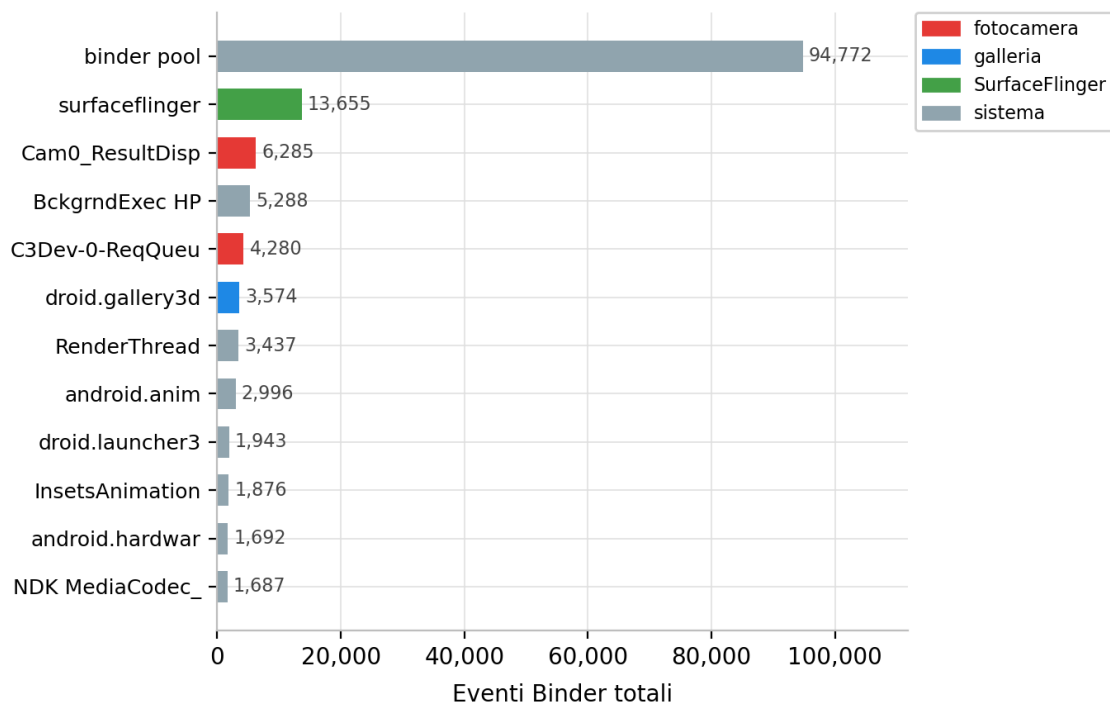


Figura 4.7: Grafo delle comunicazioni Binder durante l’anteprima della fotocamera: il canale `Cam0_ResultDisp` verso il Camera HAL è il segnale caratteristico dell’accesso all’hardware fotografico.

Nessuna comunicazione di rete esterna è associata allo scatto: la probe di invio dati registra per `droid.gallery3d` un totale di soli circa 4 KB durante l’intera sessione, esclusivamente su socket locali verso servizi di sistema.

4.3.4 Osservazioni sull’architettura osservata

Lo scenario galleria e fotocamera si distingue dalla navigazione web per l’assenza totale di traffico di rete esterno. La probe di associazione socket non registra alcun evento per l’intera sessione, e quella di accettazione connessioni rileva attività solo da processi di sistema come `init` e `netd`, mai dalle applicazioni utente. È la conferma che galleria e fotocamera operano esclusivamente su risorse locali.

Nonostante questa chiusura alla rete, il volume di comunicazione IPC è considerevole: la sessione ha prodotto 38 824 transazioni Binder per un totale di circa 17.2 MB trasferiti via IPC, a fronte di appena 4 KB inviati via socket. Anche opera-

zioni che l'utente percepisce come completamente locali generano internamente un flusso continuo di messaggi tra decine di componenti di sistema.

Il dato più interessante riguarda l'identificazione dell'accesso alla fotocamera. L'attivazione del thread `Cam0_ResultDisp` e del canale Binder verso il Camera HAL è un indicatore preciso e stabile: non compare durante l'uso della galleria, appare non appena la fotocamera viene avviata, e persiste fino alla chiusura dell'applicazione. Questo tipo di segnatura IPC potrebbe essere utilizzato per rilevare l'accesso alla fotocamera da parte di un processo, indipendentemente da qualsiasi indicatore a livello applicativo.

Categoria	Volume totale
Transazioni Binder	38 824
Traffico IPC	17.2 MB
Dati inviati via socket	4 KB
Dati ricevuti via socket	215 KB
Bind aperti	0

Tabella 4.3: Riepilogo del traffico complessivo della sessione galleria e fotocamera: 38 824 transazioni Binder e traffico di rete trascurabile.

Capitolo 5

Limiti e possibili miglioramenti

Il sistema sviluppato raggiunge gli obiettivi dichiarati nel capitolo introduttivo: permette di eseguire bpfttrace su Android, raccogliere eventi da sette probe parallele e analizzare il comportamento delle applicazioni a livello di kernel. Le sperimentazioni descritte nel capitolo precedente dimostrano che i dati prodotti sono sufficientemente precisi e ricchi da permettere ricostruzioni dettagliate di scenari reali. Rimangono tuttavia limiti concreti che ne condizionano l'applicabilità, e margini di miglioramento che potrebbero ampliarne l'utilità in contesti diversi da quello in cui è stato sviluppato.

5.1 Limiti del sistema attuale

5.1.1 Dipendenza dall'ambiente emulato

Il vincolo più significativo è che l'intero sistema è progettato per funzionare su Android Cuttlefish in esecuzione su Ubuntu, con una distribuzione Debian installata all'interno dell'emulatore. Questo ambiente è riproducibile e controllato, ma non corrisponde a un dispositivo reale. Su un dispositivo Android di produzione, l'esecuzione di bpfttrace richiederebbe accesso root, che a sua volta implica il rooting del dispositivo o la sostituzione della ROM con una versione personalizzata. Entrambe le opzioni sono impraticabili nella maggior parte dei contesti operativi reali, come documentato nella Tabella 1.1 del capitolo introduttivo.

Anche laddove l'accesso root fosse disponibile, l'ambiente Debian necessario all'esecuzione di `bpfttrace` non sarebbe presente, e installarlo su un dispositivo fisico richiederebbe una procedura significativamente più complessa di quella seguita per Cuttlefish. Il sistema è pertanto utile per analisi in ambienti di sviluppo e test, ma non è distribuibile su flotte di dispositivi reali nella forma attuale.

5.1.2 Analisi esclusivamente post-sessione

Il sistema attuale è progettato per raccogliere dati durante una sessione e analizzarli al termine. Non è possibile ricevere notifiche in tempo reale su eventi rilevanti, né interrompere automaticamente la sessione in risposta a comportamenti anomali osservati. Questo limita l'utilità del sistema in scenari di monitoraggio continuo o di risposta agli incidenti, dove la tempestività dell'informazione è importante.

5.1.3 Identificazione dei processi

Il limite di 15 caratteri del campo `comm` nel kernel Linux rende difficile l'identificazione univoca dei processi nelle probe. Come documentato nelle analisi del capitolo precedente, nomi come `ndroid.settings`, `ndroid.systemui` o `droid.gallery3d` sono troncamenti che richiedono conoscenza del sistema per essere ricondotti all'applicazione corretta. Il modulo di orchestrazione mitiga parzialmente questo problema leggendo `/proc/<pid>/cmdline` all'evento `exec`, ma questo arricchimento non è sempre disponibile per i processi già in esecuzione al momento dell'avvio delle probe.

5.2 Direzioni di sviluppo

5.2.1 Estensione a dispositivi reali

La direzione più impattante per ampliare l'applicabilità del sistema è rendere le probe eseguibili su dispositivi Android reali. Una via percorribile senza rooting del dispositivo è l'utilizzo di Android Debug Bridge (`adb`) in modalità *shell* con i privilegi elevati disponibili nelle versioni di debug del sistema operativo, distribuite per scopi

di sviluppo. In questo scenario le probe potrebbero essere caricate dall'esterno tramite `adb shell` senza richiedere modifiche permanenti al dispositivo.

Un'altra direzione è l'integrazione con ambienti di test aziendali che già operano con ROM personalizzate o dispositivi con accesso root abilitato a scopo di analisi. In questi contesti il sistema potrebbe essere distribuito come strumento di analisi della sicurezza senza le limitazioni tipiche dei dispositivi di produzione.

5.2.2 Analisi in tempo reale e sistema di allerta

L'architettura a *pipeline* descritta nel capitolo di implementazione si presta a un'estensione verso l'analisi in tempo reale. Invece di scrivere gli eventi su file `.jsonl` e analizzarli al termine della sessione, il modulo di orchestrazione potrebbe esporre un flusso di eventi a un componente di analisi continua. Quest'ultimo potrebbe applicare regole di rilevamento e generare notifiche al verificarsi di condizioni predefinite, come l'apertura di connessioni verso indirizzi IP non visti in precedenza, l'esecuzione di binari fuori dalle directory di sistema, o l'attivazione di canali Binder verso il Camera HAL da parte di un'applicazione non autorizzata.

La soglia di frequenza di campionamento delle probe attuali è sufficiente per questo tipo di analisi: come mostrato nel capitolo precedente, i picchi di attività rilevanti sono ben distinguibili dal traffico di fondo già nell'arco di pochi secondi.

5.2.3 Firme comportamentali e rilevamento di anomalie

I dati raccolti nelle sessioni di test evidenziano che le applicazioni Android producono profili di attività caratteristici e riproducibili: la galleria genera un picco Binder specifico al caricamento delle anteprime, la fotocamera attiva thread HAL identificabili, il browser produce sequenze di `bind` correlate alle richieste HTTP/2. Questi profili potrebbero essere formalizzati come firme comportamentali da confrontare con l'attività osservata in sessioni successive.

Un sistema di rilevamento basato su firme potrebbe segnalare, ad esempio, un'applicazione che attiva il canale Binder verso il Camera HAL pur non essendo un'applicazione fotografica, o un processo che apre socket verso l'esterno in assenza di

interazione utente. Questo tipo di rilevamento non richiederebbe tecniche di apprendimento automatico nella fase iniziale: le firme osservate nelle sperimentazioni sono già sufficientemente precise da definire regole basate su soglie e pattern.

5.2.4 Estensione delle probe

Le sette probe attuali coprono syscall di rete, Binder IPC e ciclo di vita dei processi. Aree di sistema attualmente non monitorate che potrebbero offrire informazioni complementari includono:

- la gestione della memoria, tramite il tracciamento di `mmap` e `brk`, che permetterebbe di correlare picchi di allocazione con l'attività delle applicazioni;
- la schedulazione dei processi, tramite i tracepoint `sched:sched_switch` e `sched:sched_wakeup`, utile per misurare la contesa di CPU tra componenti di sistema;
- le operazioni di lettura e scrittura su file, tramite `read` e `write`, per quantificare il volume di I/O su storage e non solo gli accessi rilevati da `openat`.

L'architettura modulare del sistema facilita l'aggiunta di nuove probe senza modificare i componenti esistenti: è sufficiente implementare il nuovo script `bpfttrace`, registrarlo nel file di configurazione condiviso, e il modulo di orchestrazione e l'interfaccia utente lo renderanno automaticamente disponibile.

5.2.5 Interfaccia di visualizzazione

Il formato attuale dell'output è un report testuale generato dal modulo di analisi. Per sessioni lunghe o con molte probe attive simultaneamente, la lettura dei report diventa onerosa. Un'interfaccia di visualizzazione basata su linea temporale, anche semplice come un file HTML generato staticamente, permetterebbe di sovrapporre gli eventi delle diverse probe su un asse temporale comune e identificare visivamente le correlazioni tra probe. La disponibilità dei dati in formato JSON strutturato, già prodotto dal modulo di analisi, renderebbe questo sviluppo relativamente agevole senza modifiche al nucleo del sistema.

Capitolo 6

Conclusioni

Questa tesi ha affrontato il problema dell'utilizzo di bpftrace per il tracing di applicazioni Android, partendo dalla constatazione che gli strumenti di osservabilità tipici dei sistemi Linux non sono direttamente disponibili su Android senza adattamenti significativi. La soluzione proposta, basata sull'esecuzione di bpftrace all'interno di una distribuzione Debian installata nell'emulatore Cuttlefish, ha dimostrato che è possibile raccogliere dati di sistema a livello kernel su Android senza modificare il sistema operativo.

Il sistema sviluppato comprende sette probe bpftrace parallele, un modulo di orchestrazione in Python che coordina la raccolta degli eventi, un modulo di analisi post-sessione e un'interfaccia utente bash. Le probe coprono il ciclo di vita dei processi, le syscall di accesso a file e rete, le transazioni IPC Binder e il volume di dati scambiati via socket. L'architettura a pipeline consente di attivare solo il sottoinsieme di probe necessario per lo scenario di interesse, limitando il volume di dati prodotti e l'overhead sul sistema monitorato.

Le sperimentazioni condotte su tre scenari distinti hanno mostrato che i dati raccolti dal sistema sono sufficientemente precisi da permettere ricostruzioni dettagliate del comportamento delle applicazioni a livello di sistema operativo.

In tutti e tre gli scenari, le correlazioni più significative sono emerse dal confronto tra probe diverse, non dall'analisi di una singola fonte di dati. Questo risultato conferma la premessa del lavoro: un sistema di monitoraggio parallelo e multidimensionale permette di osservare il comportamento di Android in modo qualitativamente

diverso rispetto a strumenti che operano su una sola categoria di eventi.

I limiti del sistema, discussi nel capitolo precedente, riguardano principalmente la dipendenza dall'ambiente emulato e l'assenza di analisi in tempo reale. Entrambi sono limiti architetturali che non invalidano il contributo della tesi, ma definiscono il confine dell'applicabilità attuale: il sistema è uno strumento di analisi e ricerca, non un agente di monitoraggio distribuibile su dispositivi di produzione nella forma attuale.

Il contributo principale di questo lavoro è dimostrare che `bpfftrace`, con l'infrastruttura appropriata, può essere utilizzato per ottenere una visibilità profonda sul comportamento di Android a livello kernel. I dati raccolti hanno permesso di osservare meccanismi interni del sistema operativo, come l'architettura di isolamento del renderer Chromium, la pipeline di elaborazione dei frame della fotocamera e la sequenza di riconnessione WiFi, che non sarebbero accessibili attraverso le API applicative o gli strumenti di profilazione standard. Questo apre prospettive concrete per l'utilizzo di tecniche eBPF nell'analisi della sicurezza delle applicazioni Android, nella verifica del comportamento di componenti di sistema e nel rilevamento di anomalie basato su firme comportamentali a livello kernel.

Bibliografia

- [1] Android Open Source Project. *Architettura della piattaforma Android*. 2024. URL: <https://source.android.com/docs/core/architecture?hl=it> (visitato il 10/03/2026).
- [2] Android Open Source Project. *Binder IPC*. 2024. URL: <https://source.android.com/docs/core/architecture/hidl/binder-ipc>.
- [3] Android Open Source Project. *Camera HAL3*. 2024. URL: <https://source.android.com/docs/core/camera/camera3> (visitato il 12/03/2026).
- [4] Android Open Source Project. *Cuttlefish Virtual Android Devices*. 2024. URL: <https://source.android.com/docs/setup/create/cuttlefish>.
- [5] Android Open Source Project. *SurfaceFlinger and WindowManager*. 2024. URL: <https://source.android.com/docs/core/graphics/surfaceflinger-windowmanager> (visitato il 12/03/2026).
- [6] Apache Software Foundation. *Apache Spark*. 2024. URL: <https://spark.apache.org>.
- [7] Matt Bishop e Michael Dilger. «Checking for Race Conditions in File Accesses». In: *USENIX Computing Systems*. Vol. 9. 2. 1996.
- [8] Blagin. *Fork, Exec, Exit, Wait*. 2024. URL: <https://blagin.ru/linux-dlya-nachinayushhix/fork-exec-exit-wait/> (visitato il 10/03/2026).
- [9] bpftrace contributors. *bpftrace Reference Guide*. 2024. URL: https://github.com/bpftrace/bpftrace/blob/master/docs/reference_guide.md.
- [10] John Doe e John Smith. «Paper title». In: *Proceedings of the Big Conf*. Giu. 2017, pp. 185–200. DOI: <https://doi.org/doi/reference/number>.

- [11] Stephen Dolan. *jq — Command-line JSON processor*. 2024. URL: <https://jqlang.github.io/jq/>.
- [12] Ralph Droms. *Dynamic Host Configuration Protocol*. RFC 2131. Internet Engineering Task Force, 1997. URL: <https://datatracker.ietf.org/doc/html/rfc2131>.
- [13] ebpf.io. *What is eBPF?* 2024. URL: <https://ebpf.io/what-is-ebpf/>.
- [14] Mattia Ferrari. *android-ebpf-monitor*. 2026. URL: <https://github.com/tiaferra/android-ebpf-monitor> (visitato il 11/03/2026).
- [15] Matt Fleming. *A thorough introduction to eBPF*. 2017. URL: <https://lwn.net/Articles/740157/>.
- [16] Google LLC. *Android Open Source Project*. 2024. URL: <https://source.android.com> (visitato il 11/03/2026).
- [17] IBM. *What is data persistence?* 2024. URL: <https://www.ibm.com/topics/data-persistence>.
- [18] JSON Lines contributors. *JSON Lines*. 2024. URL: <https://jsonlines.org>.
- [19] Linux kernel contributors. *perf: Linux profiling with performance counters*. 2024. URL: <https://perf.wiki.kernel.org>.
- [20] Linux man-pages project. *ptrace(2) — Linux manual page*. 2024. URL: <https://man7.org/linux/man-pages/man2/ptrace.2.html>.
- [21] Bikash Nagarkoti. *Linux Architecture*. 2021. URL: https://www.researchgate.net/figure/Linux-Architecture_fig1_349554547 (visitato il 10/03/2026).
- [22] Eric S. Raymond. *The Art of Unix Programming*. Addison-Wesley, 2003. URL: <http://www.catb.org/esr/writings/taoup/>.
- [23] Steven Rostedt. *ftrace — Function Tracer*. 2024. URL: <https://www.kernel.org/doc/html/latest/trace/ftrace.html>.
- [24] StatCounter. *Mobile Operating System Market Share Worldwide*. 2024. URL: <https://gs.statcounter.com/os-market-share/mobile/worldwide>.

- [25] strace contributors. *strace — system call tracer*. 2024. URL: <https://strace.io>.
- [26] The Chromium Projects. *Multi-process Architecture*. 2024. URL: <https://www.chromium.org/developers/design-documents/multi-process-architecture/> (visitato il 12/03/2026).
- [27] The Debian Project. *Getting Debian*. 2024. URL: <https://www.debian.org/distrib/> (visitato il 11/03/2026).
- [28] The Linux Kernel Organization. *Task Structure — The Linux Kernel documentation*. 2024. URL: <https://www.kernel.org/doc/html/latest/>.
- [29] The Linux Kernel Organization. *The Linux Kernel*. 2024. URL: <https://www.kernel.org>.
- [30] The pandas development team. *pandas — Python Data Analysis Library*. 2024. URL: <https://pandas.pydata.org>.
- [31] Martin Thomson e Cory Benfield. *HTTP/2*. RFC 9113. Internet Engineering Task Force, 2022. URL: <https://datatracker.ietf.org/doc/html/rfc9113>.